PROYECTO FINAL

INFORME

FlashStore: tienda en línea de alta concurrencia

Curso: QA · Aseguramiento de la Calidad del Software

Ingeniería en Sistemas · UMG, Campus Cobán

Catedrático: Ing. Byron Alexander Caal Figueroa

Fecha: 5 de noviembre de 2026

Presentado por:

María Guadalupe Sagui
Carné 4090-22-5591

Elder Fernando Chub Cac
Carné 4090-22-5007

1. De qué se trata el proyecto

FlashStore es una tienda en línea de ofertas relámpago: hay pocas unidades de un producto y muchas personas tratan de comprarlo al mismo tiempo. Lo más peligroso en una tienda así es vender más de lo que hay en bodega (sobreventa).

Nuestro trabajo fue construir la tienda, subirla a la nube y, sobre todo, probarla de muchas formas. Al final, un pipeline de Jenkins revisa todo solo y decide si la versión está lista para salir (GO) o no (NO-GO).

El resultado más importante:

Ejecución	Decisión	Por qué
Build #8	NO-GO	Con la estrategia "Unsafe" se vendieron más unidades de las que había. Era a propósito, para demostrar que el pipeline lo detecta.
Build #9	NO-GO	Archivos con mal formato, un posible secreto y 12 alertas de librerías. Las revisamos una por una.
Build #10	GO	Todo en verde: la versión cumple los criterios de salida.

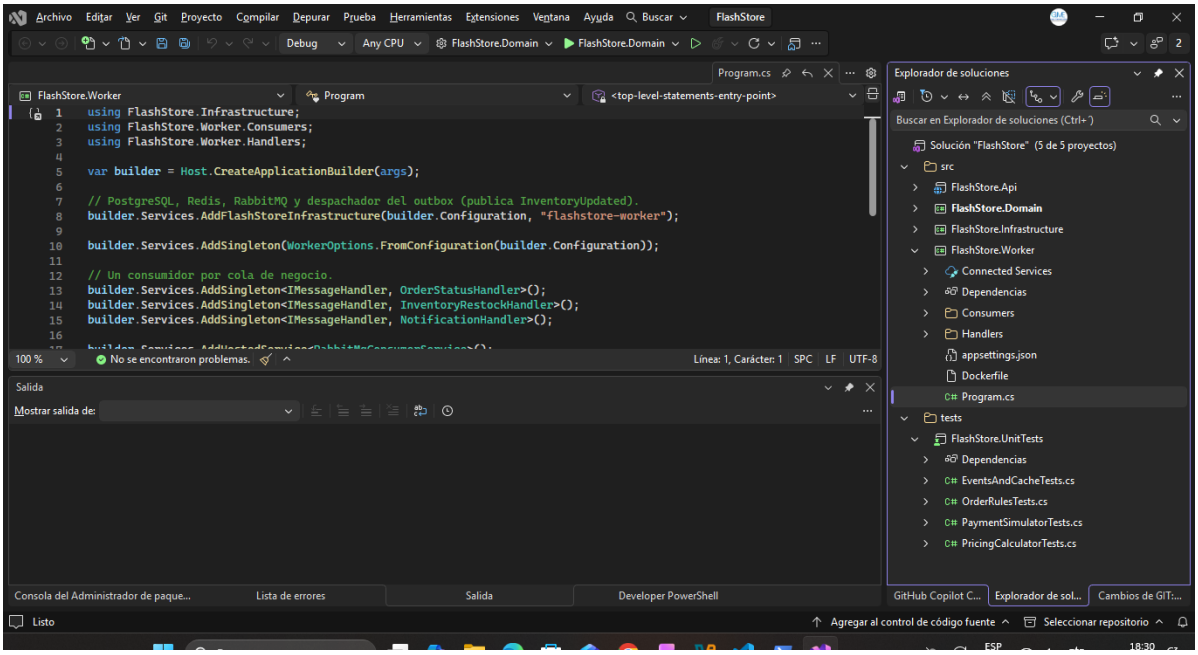
2. Herramientas que usamos

Para qué	Herramienta
Programar la tienda	.NET 8 (API), React (PWA), PostgreSQL, Redis y RabbitMQ
Ambiente local	VirtualBox con Ubuntu, Docker, Visual Studio y VS Code
Trabajo en equipo	GitHub: issues, ramas, pull requests y revisiones
Nube	Google Cloud, con Terraform (crea la infraestructura) y Ansible (instala la app)
Automatización	Jenkins con dos pipelines
Pruebas	xUnit, Insomnia, Cypress, Maestro, k6 y el CLI propio flashstore
Seguridad y calidad	gitleaks, checkov, Semgrep, SonarQube, trivy, OWASP ZAP, OWASP Dependency-Check, npm audit, ESLint
Usabilidad	Cuestionario SUS y Lighthouse

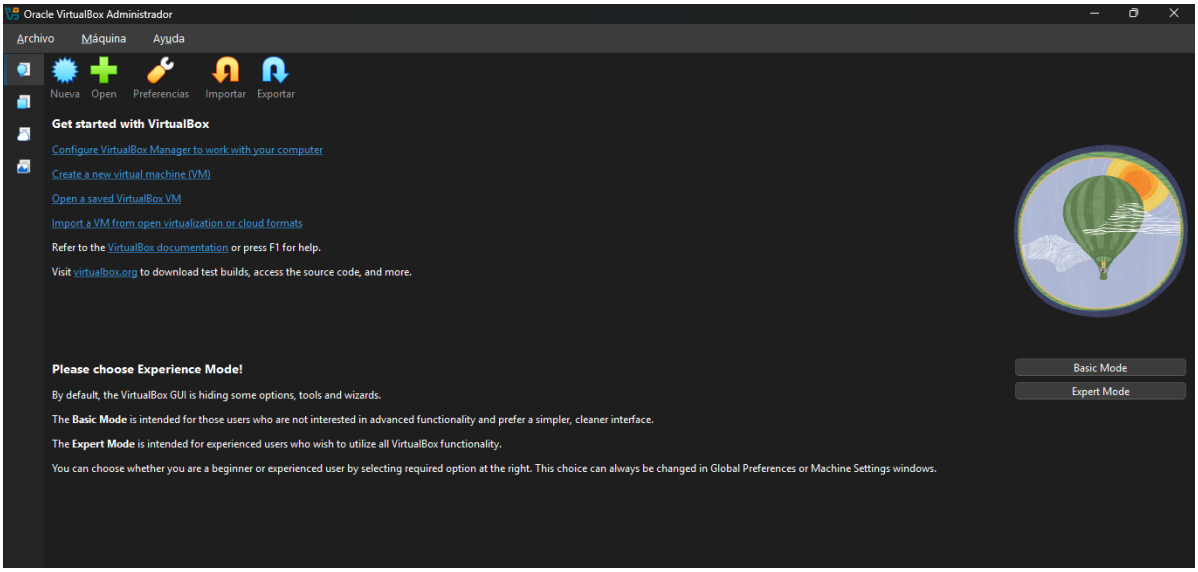
3. Preparación del ambiente

Primero armamos todo en la laptop: la solución en Visual Studio, la máquina virtual con Ubuntu y los contenedores de Docker.

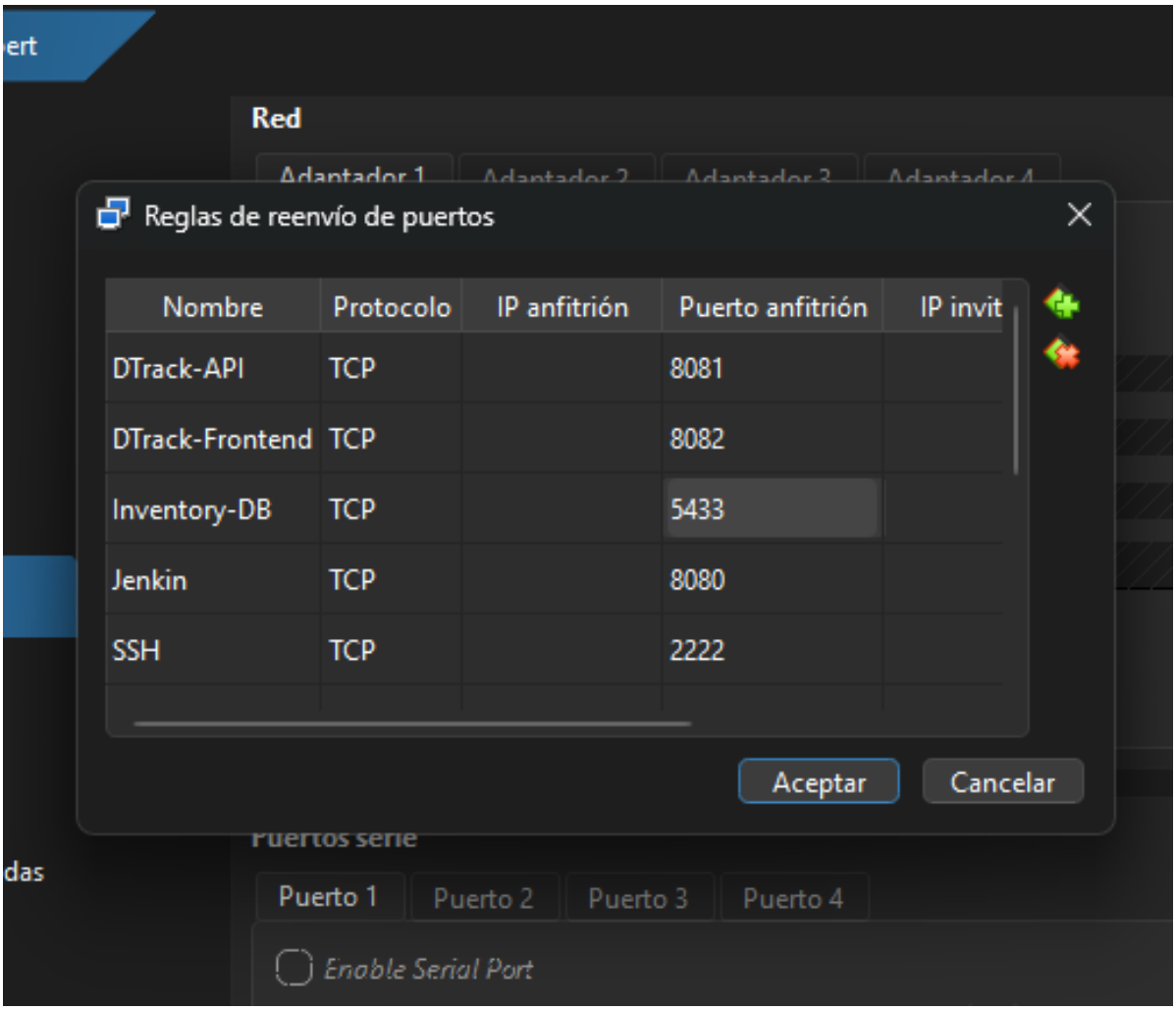
La solución de FlashStore abierta en Visual Studio. Aquí están los proyectos de la API, el dominio, la infraestructura y el Worker.



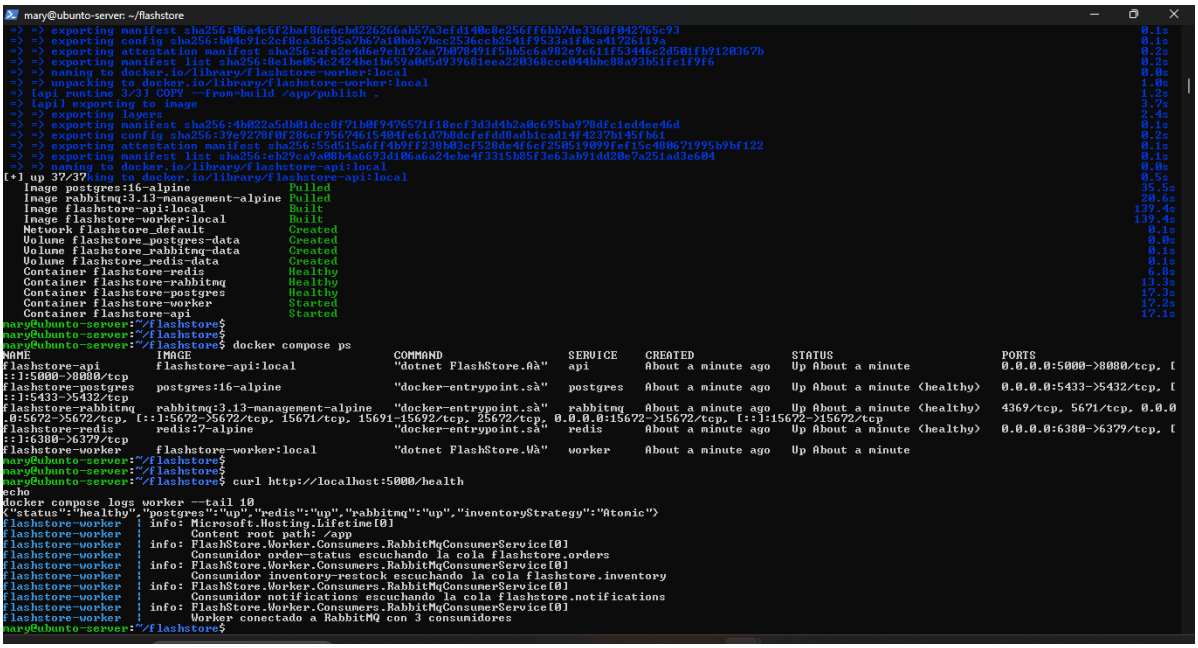
Instalamos VirtualBox para tener una máquina Ubuntu donde corren Docker y, más adelante, Jenkins.



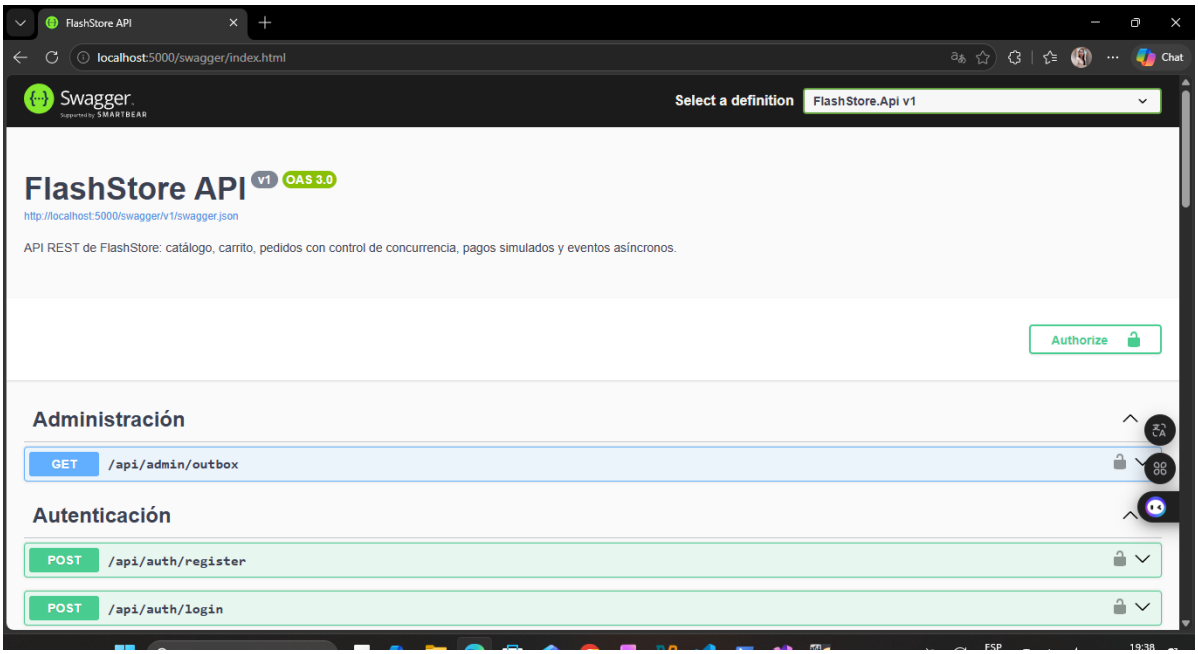
Abrimos puertos de la máquina virtual hacia la laptop: SSH en el 2222 y Jenkins en el 8080, entre otros.



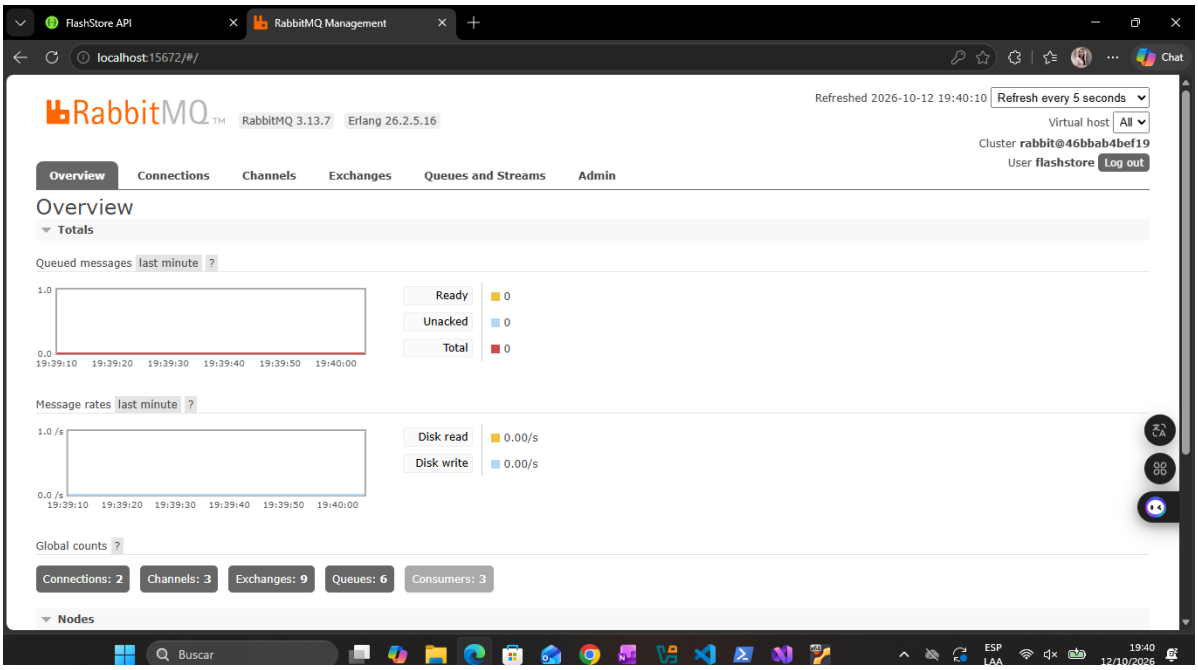
Docker construyendo las imágenes de la tienda (API, Worker y web) dentro de Ubuntu.



La API ya funcionando. Swagger muestra todos los servicios: autenticación, catálogo, carrito, pedidos y pagos.



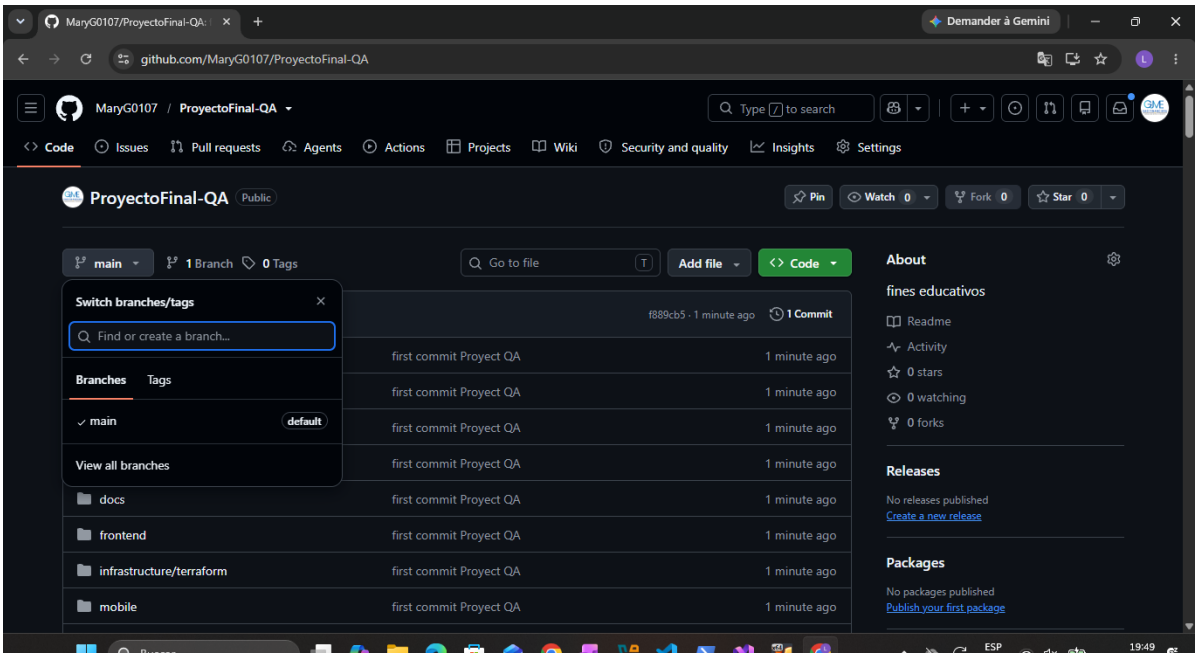
RabbitMQ funcionando. Es la cola de mensajes que usan los pedidos y los pagos.



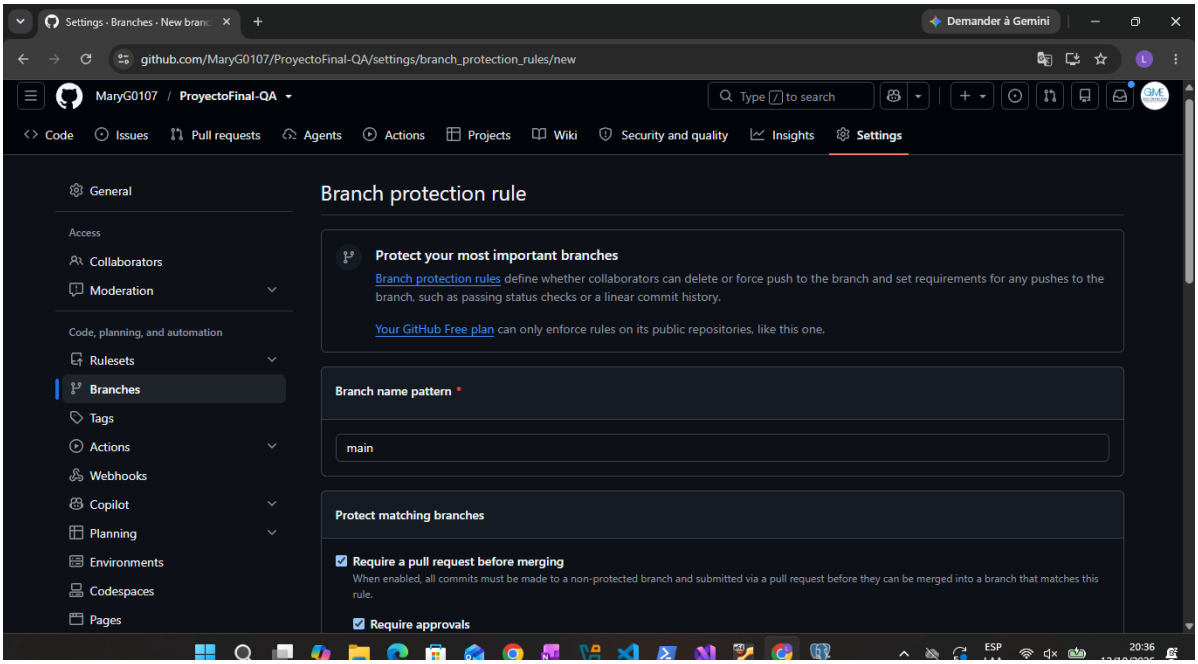
4. Trabajo en equipo con GitHub

Cada tarea siguió el mismo camino: se abre un issue, se crea una rama, se hace un pull request, la otra persona lo revisa y aprueba, y luego se une al proyecto.

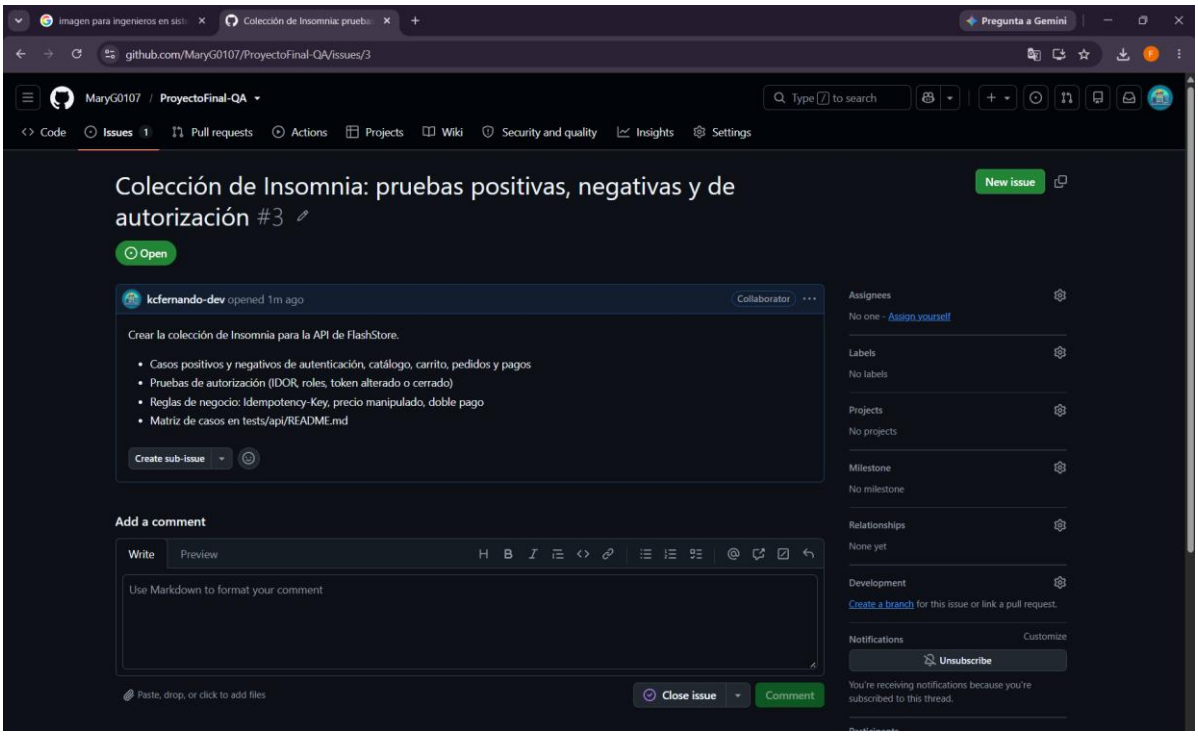
El repositorio del proyecto en GitHub con sus carpetas.



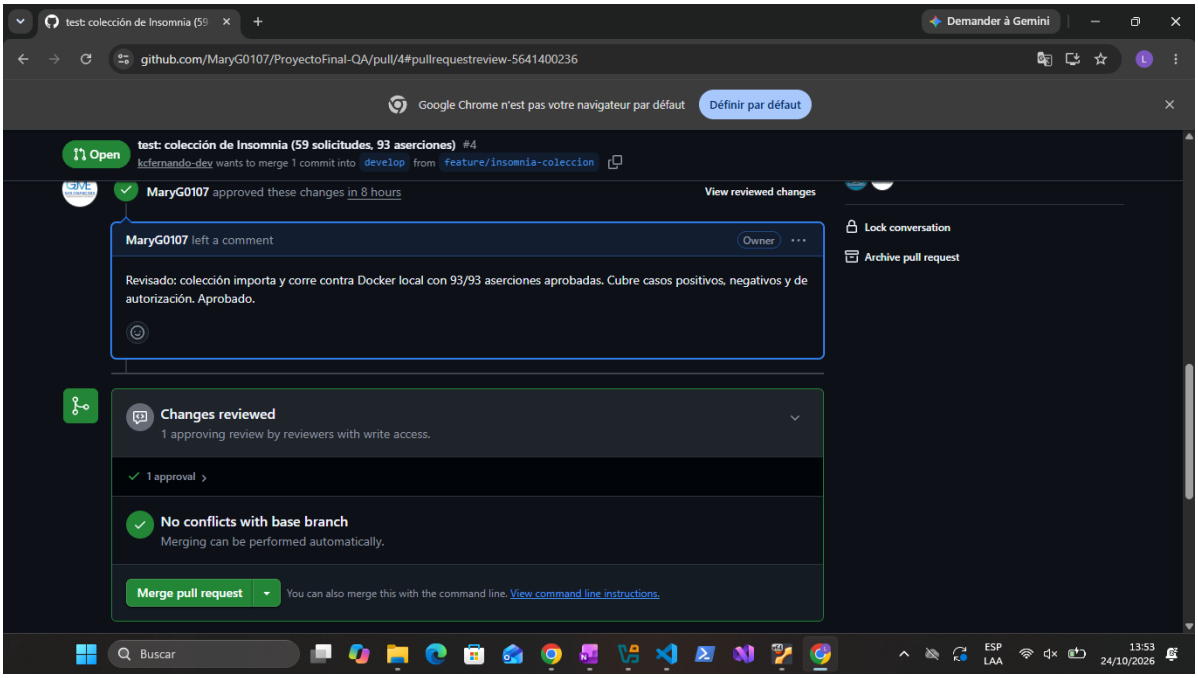
Protegimos la rama main: nadie puede subir cambios directo, todo tiene que pasar por un pull request aprobado.



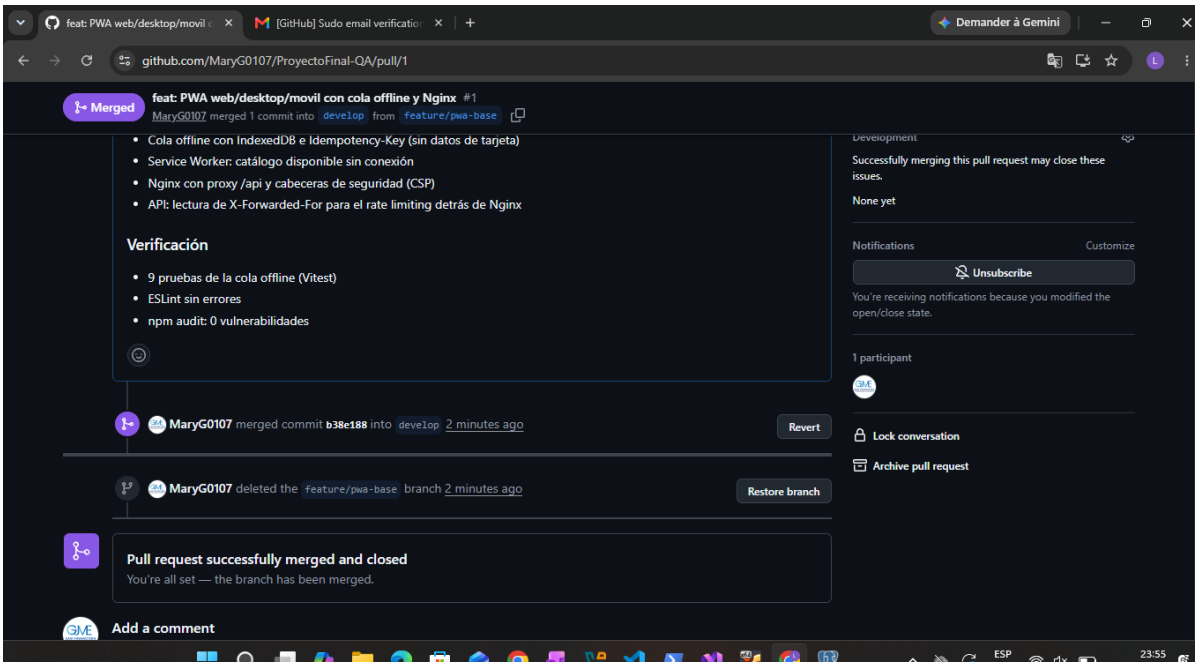
Ejemplo de issue: la colección de pruebas de Insomnia, con lo que había que hacer.



El compañero revisa el pull request y lo aprueba antes de unirlo.



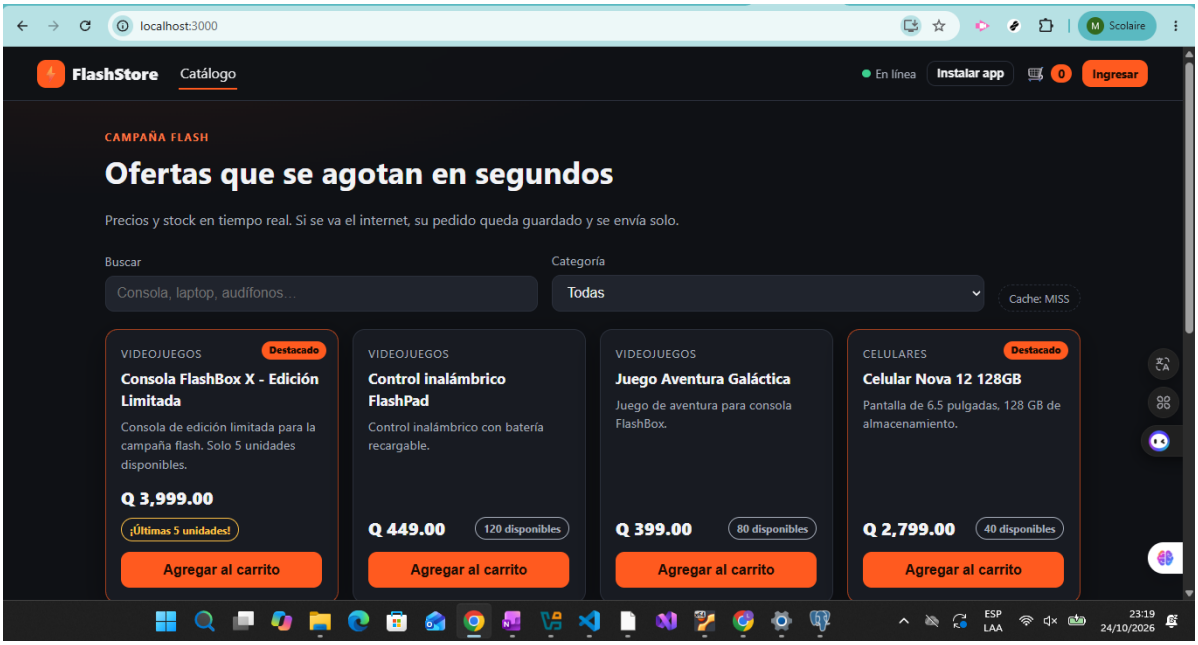
Pull request unido. Así fue con cada fase del proyecto.



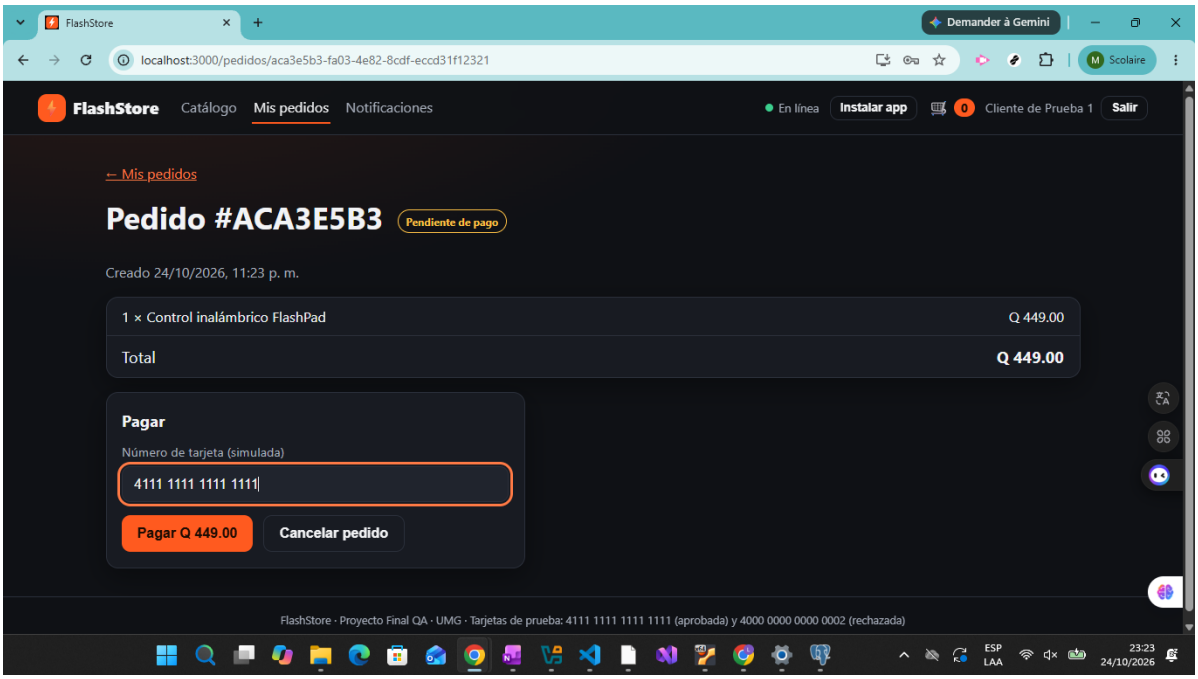
5. La tienda funcionando

Probamos a mano los flujos principales, como lo haría un cliente.

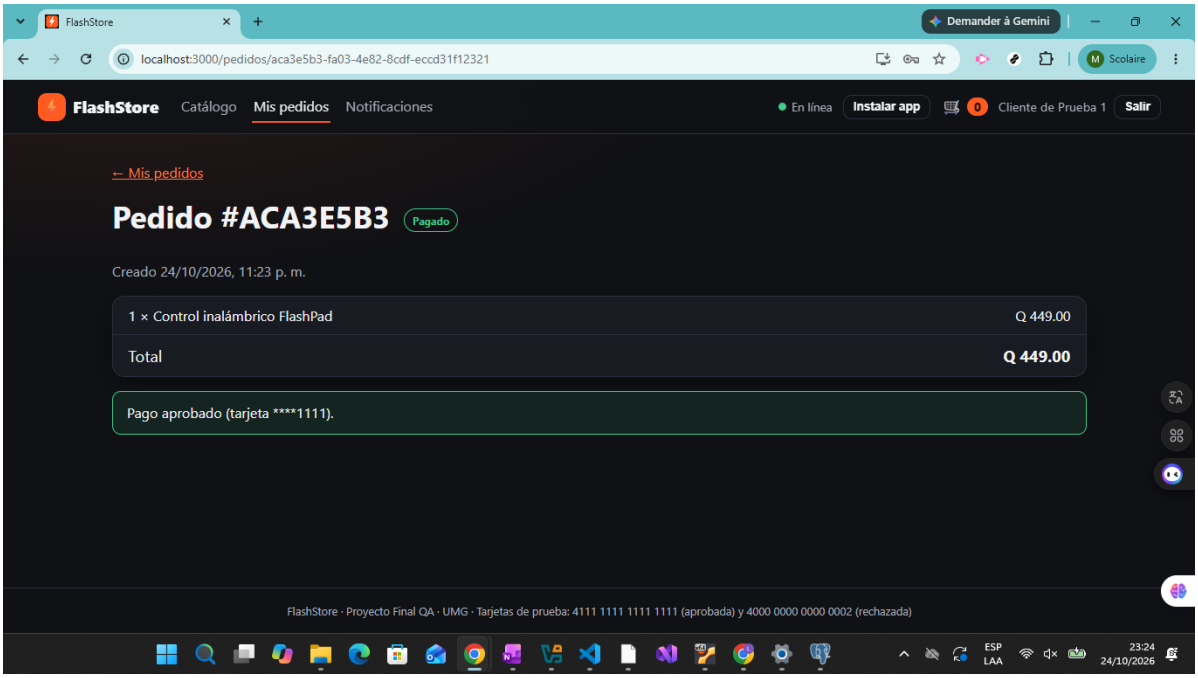
El catálogo con las ofertas, el precio y las unidades disponibles de cada producto.



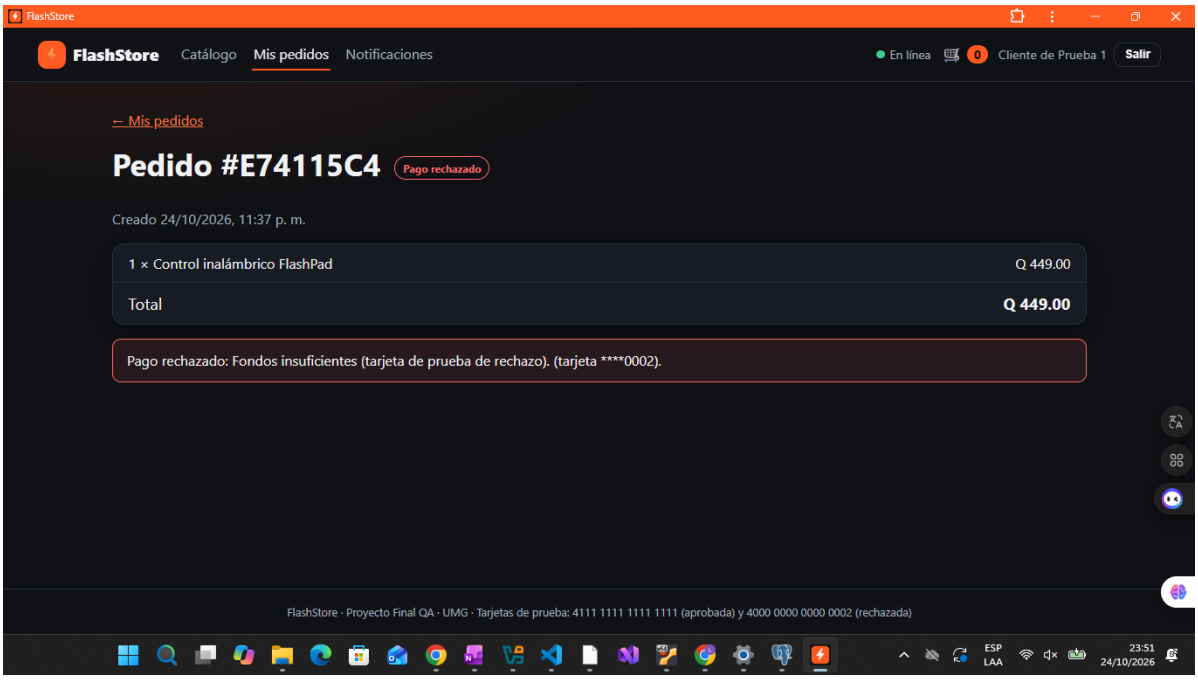
Pantalla de pago del pedido con la tarjeta de prueba.



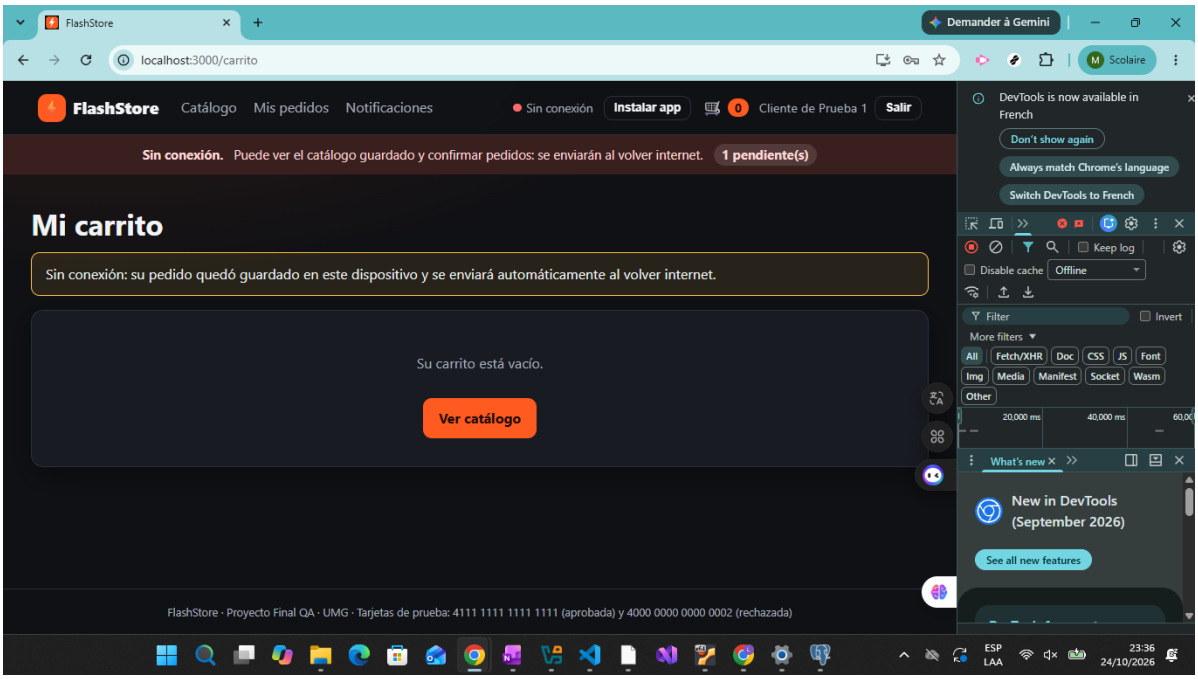
El pago fue aprobado y el pedido quedó como "Pagado".



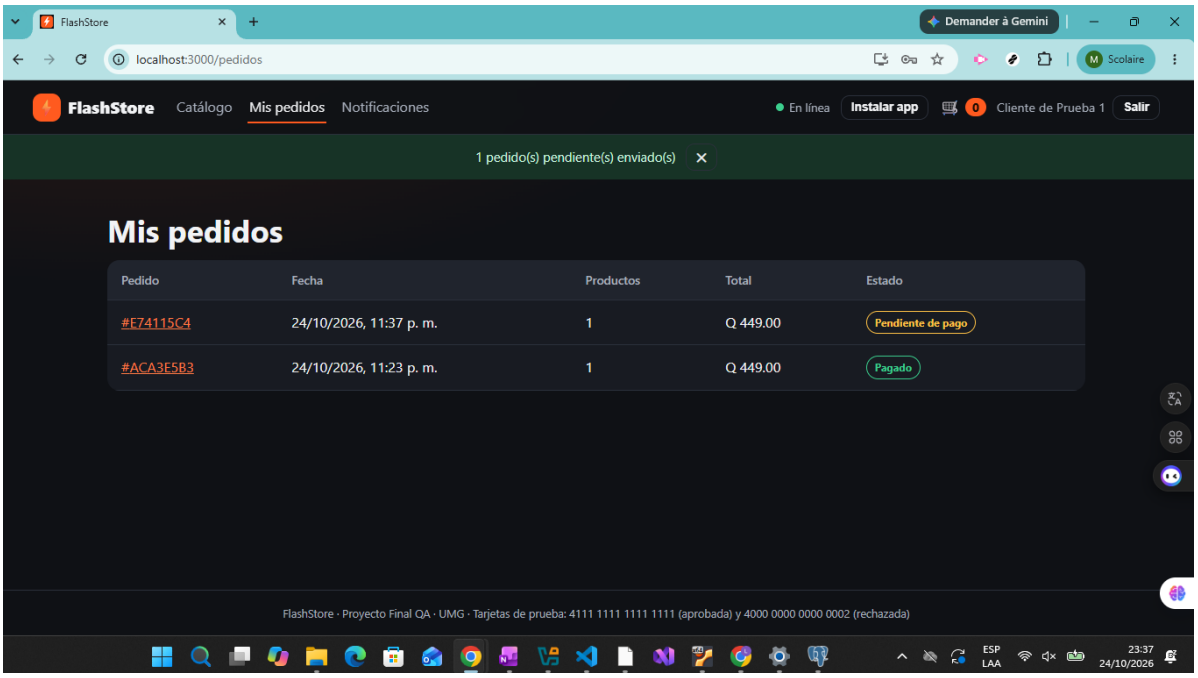
Con la tarjeta de rechazo el sistema dice claramente que el pago fue rechazado por fondos insuficientes.



Modo sin internet: el pedido se guarda en el dispositivo y se envía solo cuando vuelve la conexión.



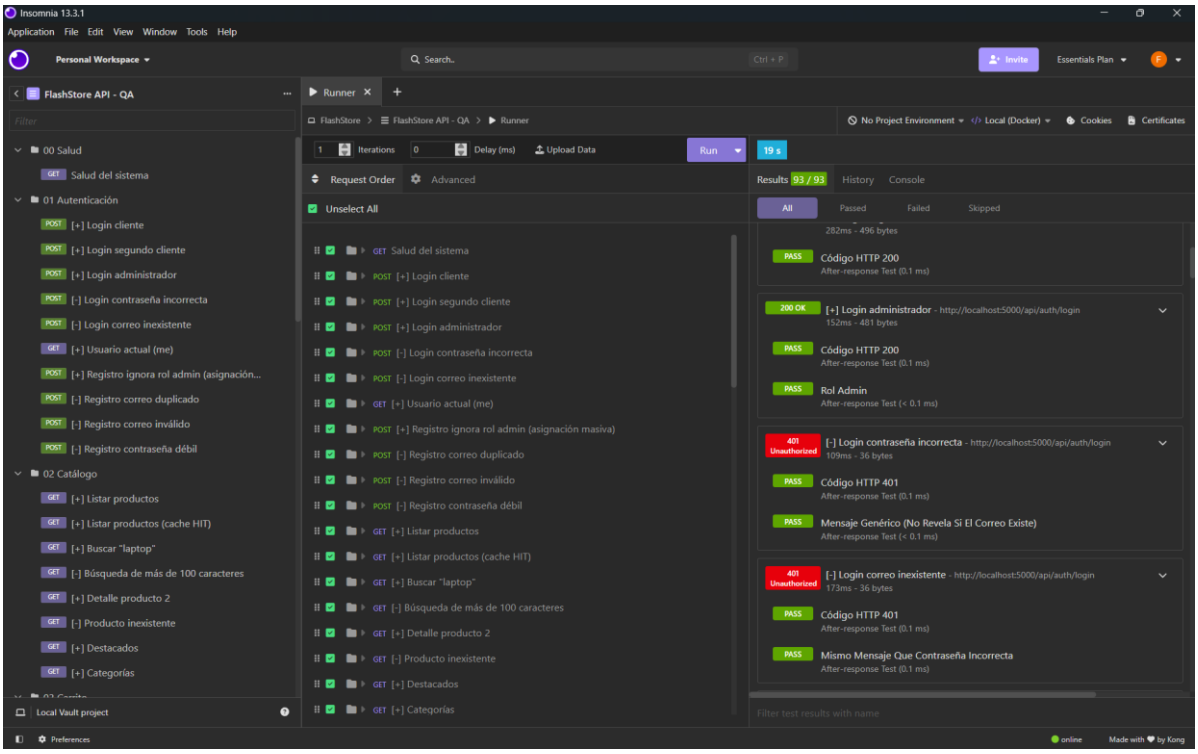
Al volver la conexión, el pedido pendiente se envió y aparece en "Mis pedidos".



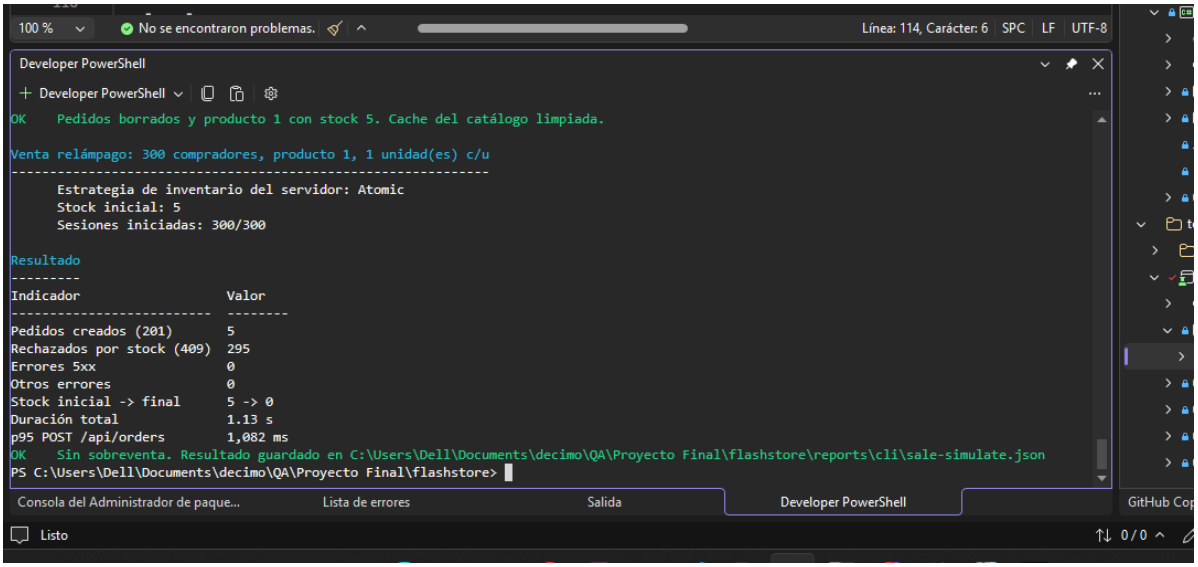
6. Pruebas de API, CLI y E2E

Automatizamos las pruebas para no tener que hacerlas a mano cada vez.

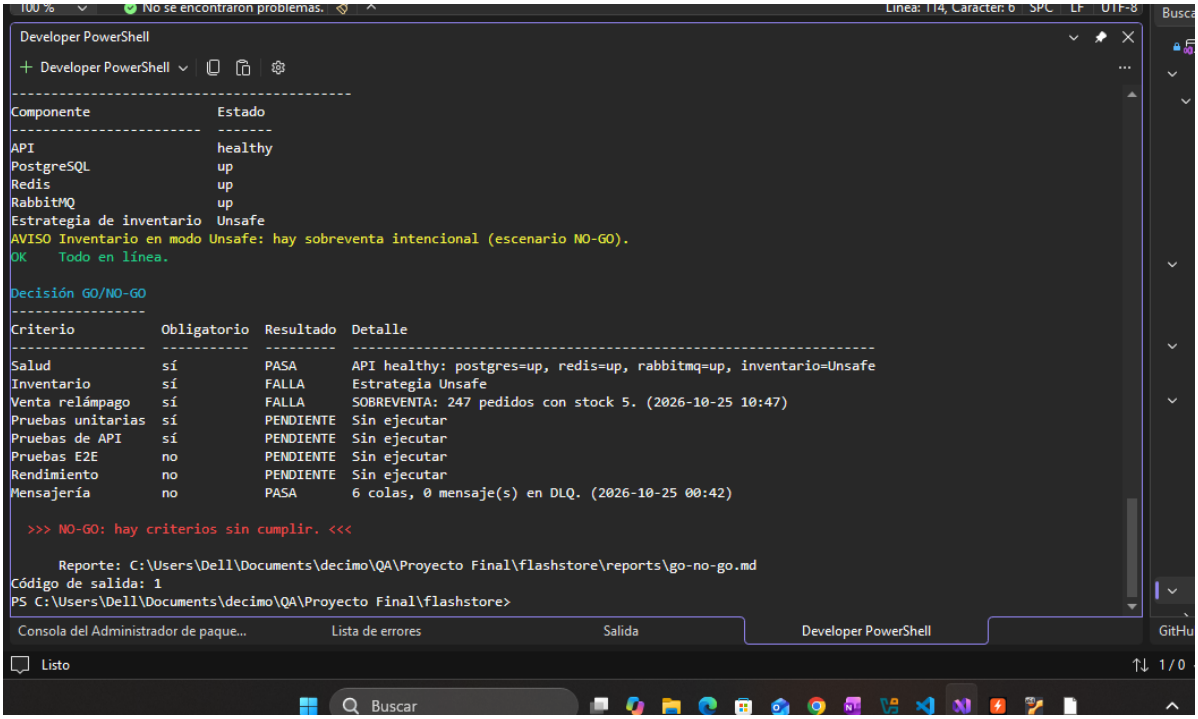
Insomnia ejecutando la colección de pruebas de la API: casos que deben funcionar y casos que deben fallar (por ejemplo, contraseña incorrecta).



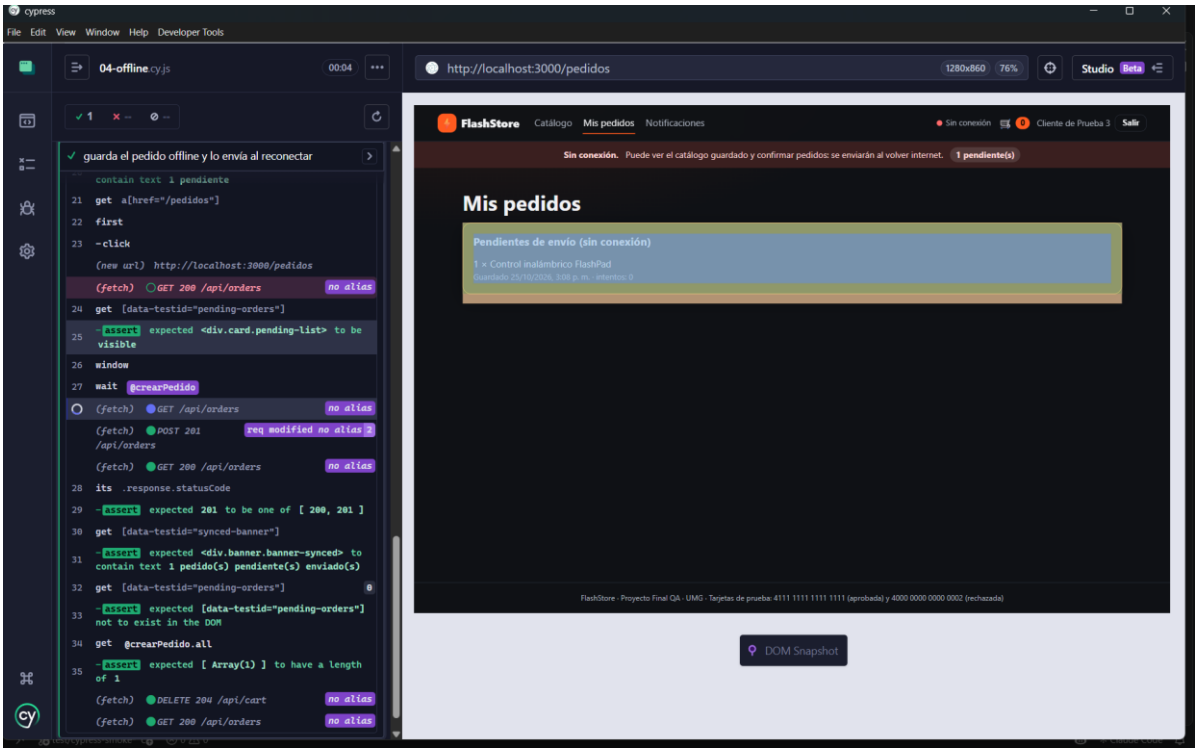
Nuestro CLI simulando la venta relámpago: 300 compradores para 5 unidades.



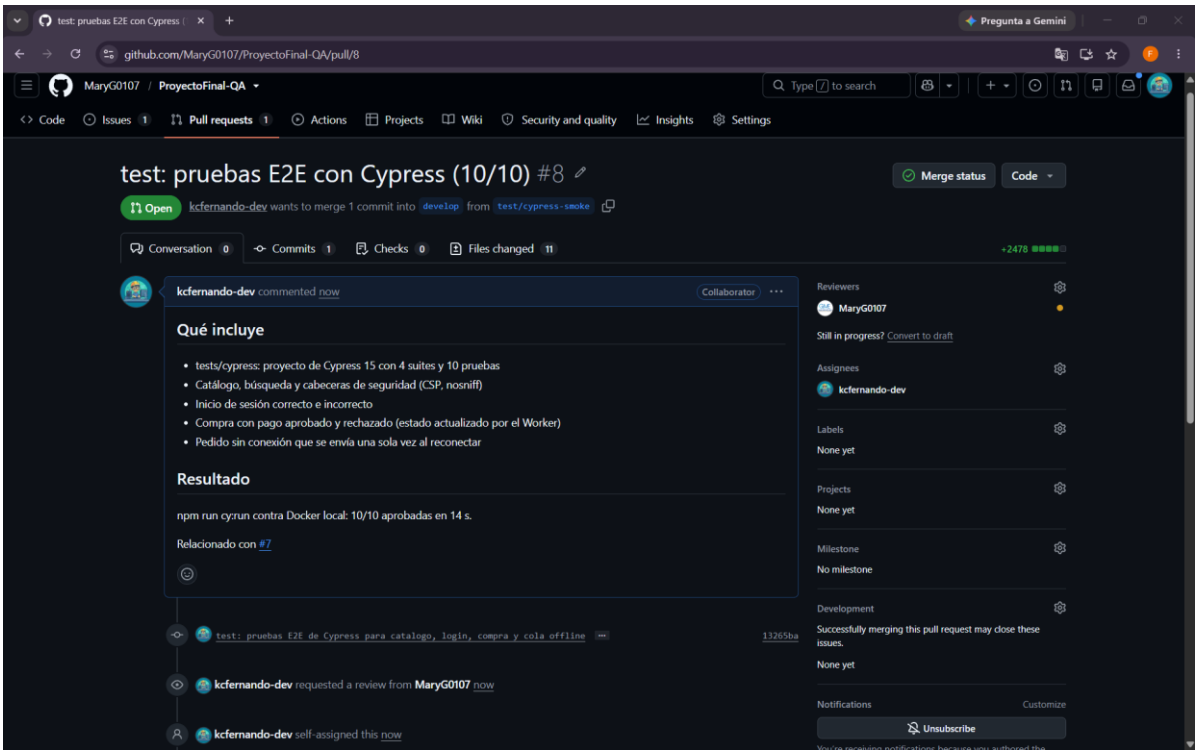
El CLI decide NO-GO porque la estrategia "Unsafe" provocó sobreventa. Era justo lo que queríamos demostrar.



Cypress probando la tienda en el navegador, aquí el caso de pedido sin internet.



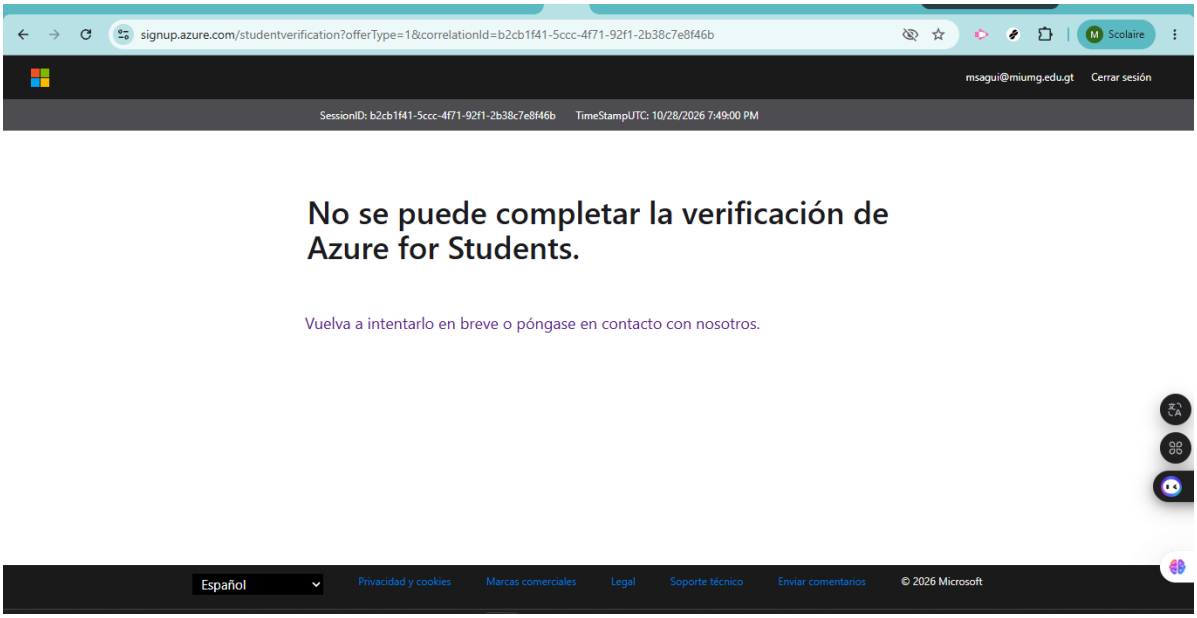
Pull request de las pruebas E2E con Cypress: 10 de 10 aprobadas.



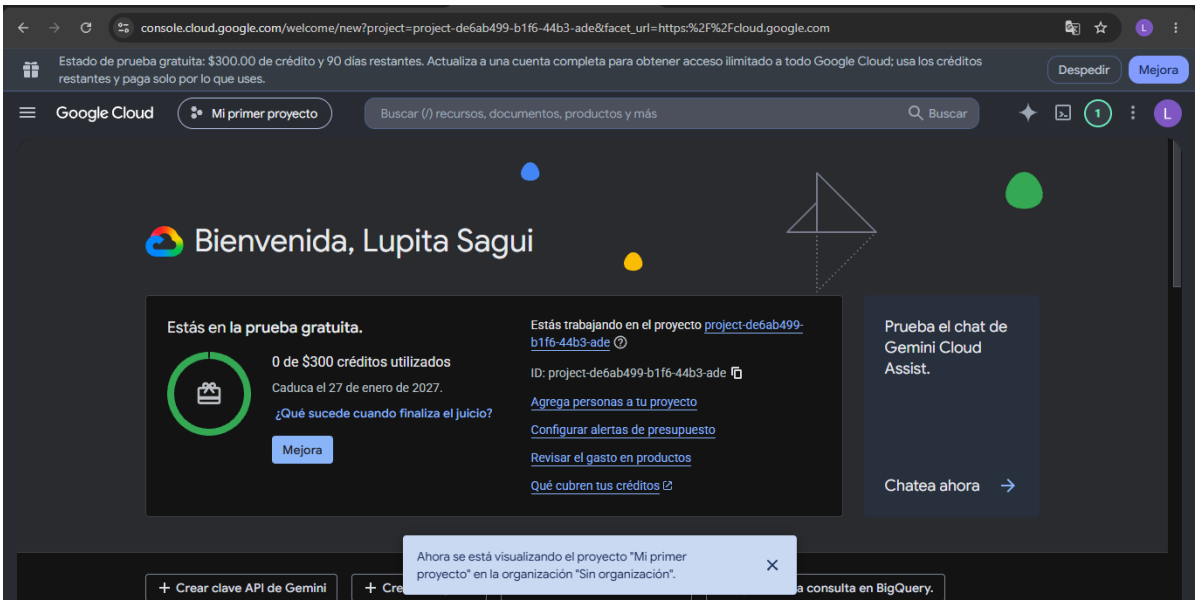
7. Subir la tienda a la nube

Azure for Students no se pudo activar con ninguna de las dos cuentas, así que cambiamos a Google Cloud.

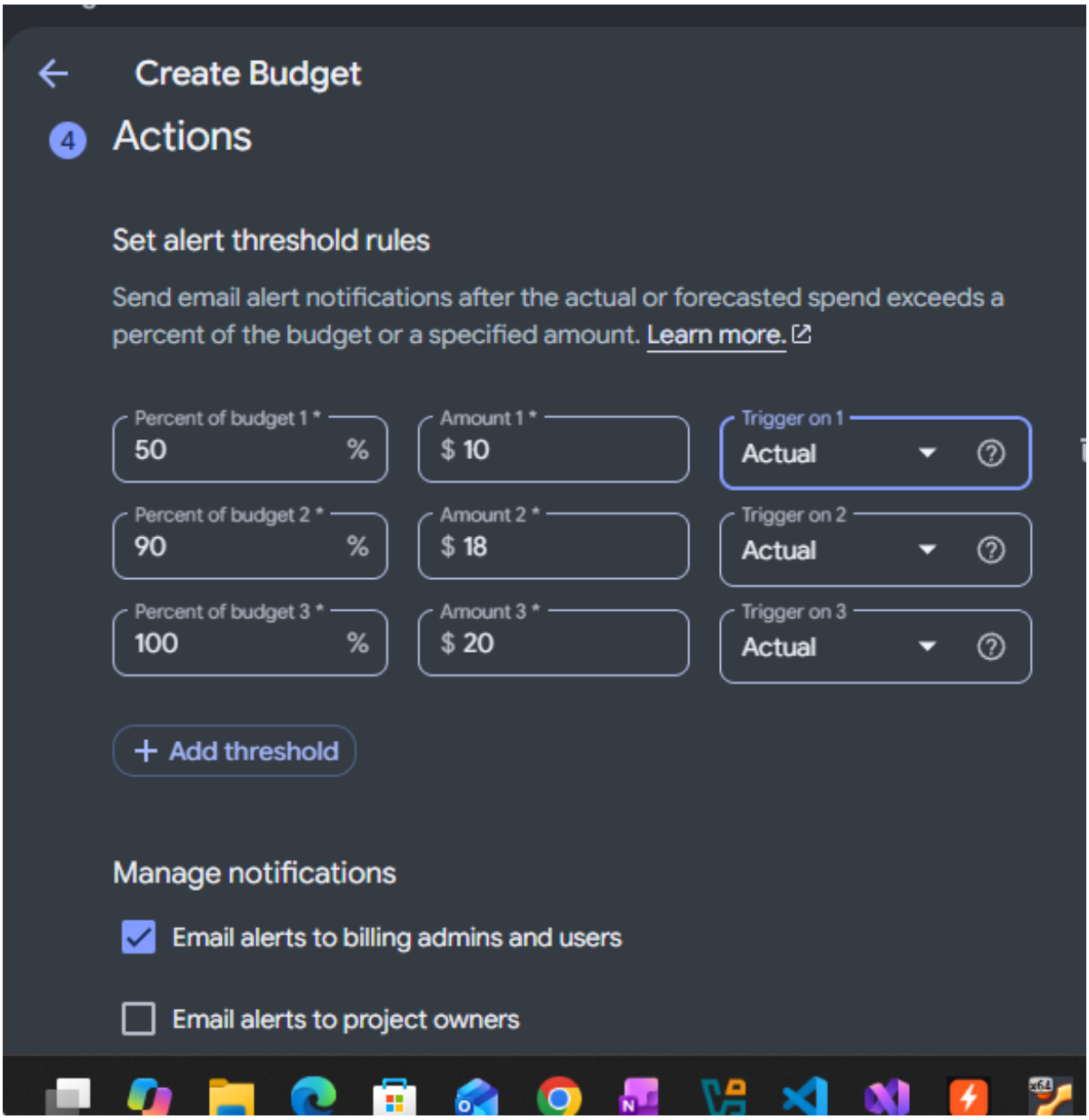
El error de Azure: no se pudo completar la verificación de estudiante.



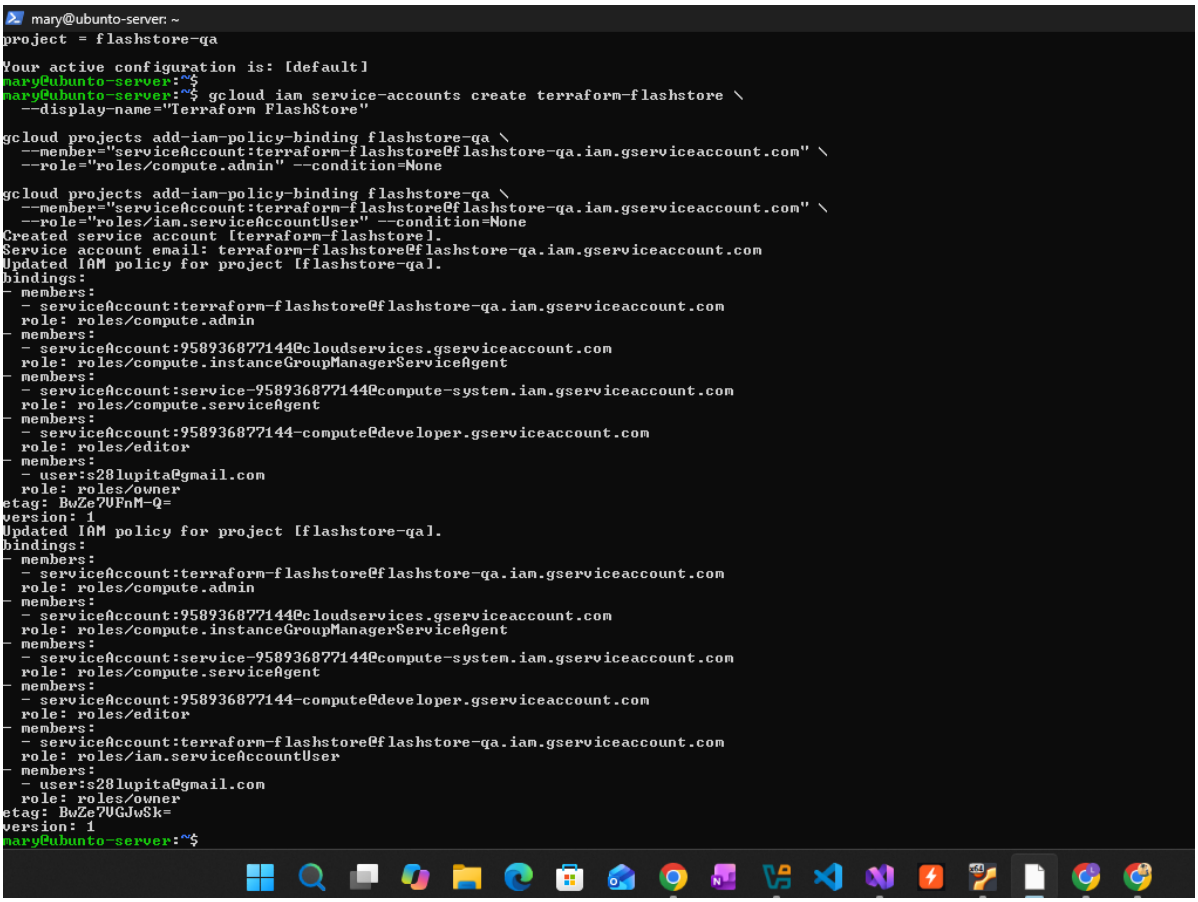
Cuenta de Google Cloud creada con el crédito de prueba.



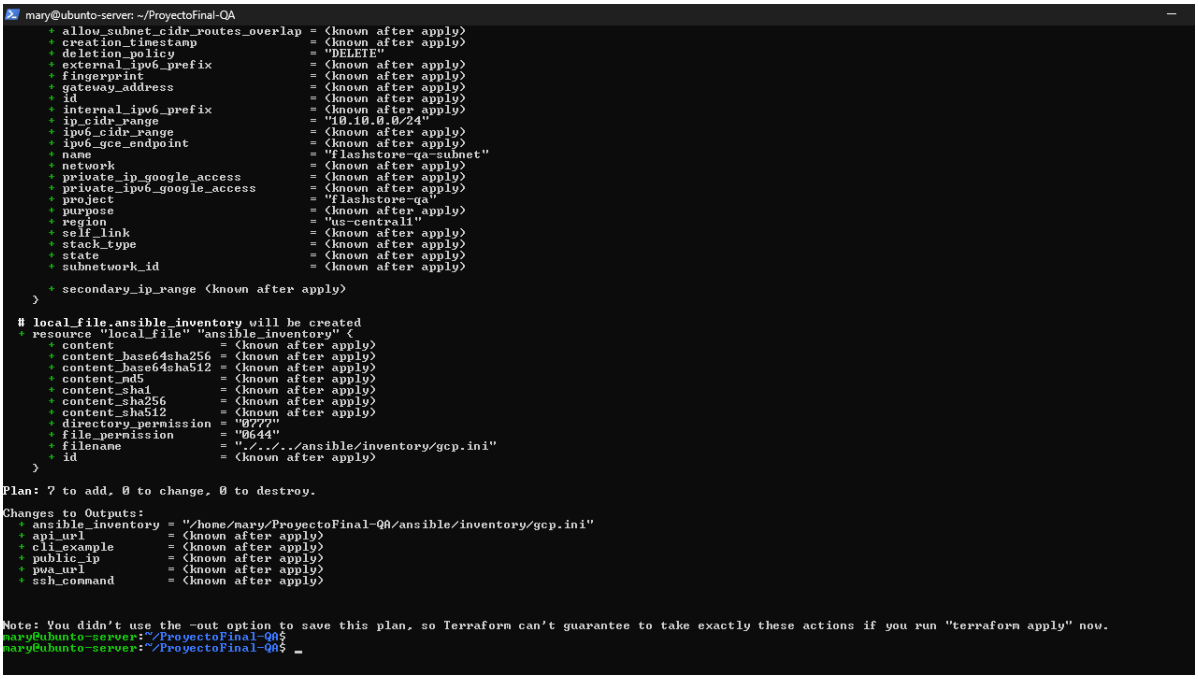
Pusimos alertas de presupuesto para que nos avise antes de gastar de más.



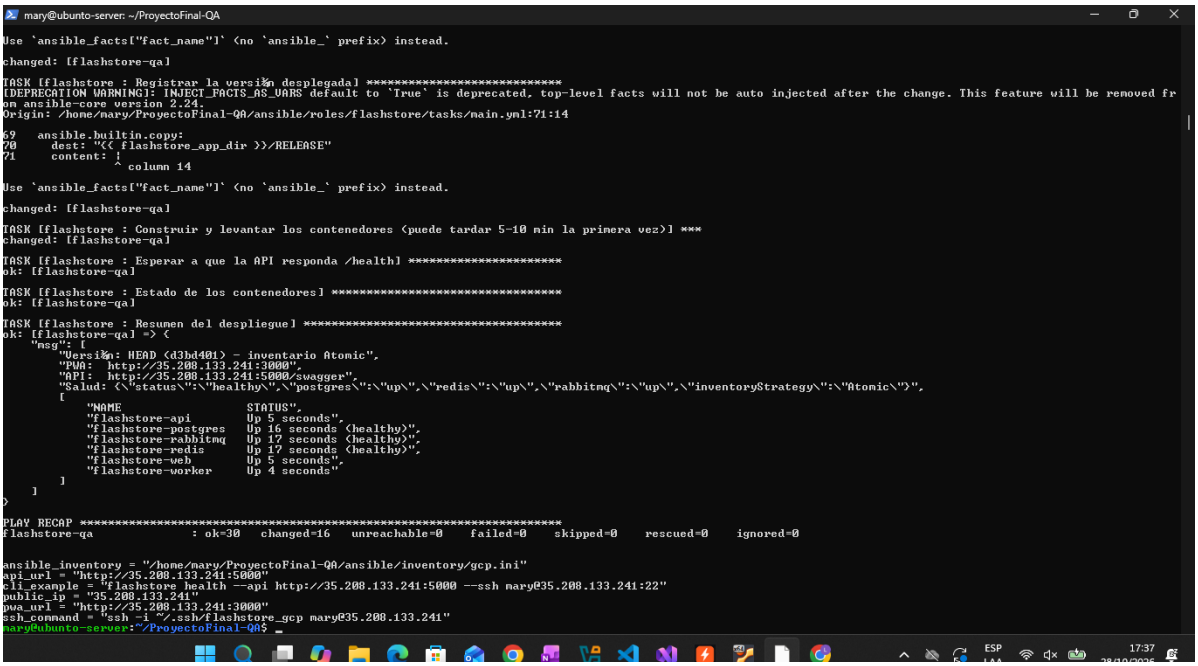
Creamos una cuenta de servicio solo para Terraform, con los permisos justos.



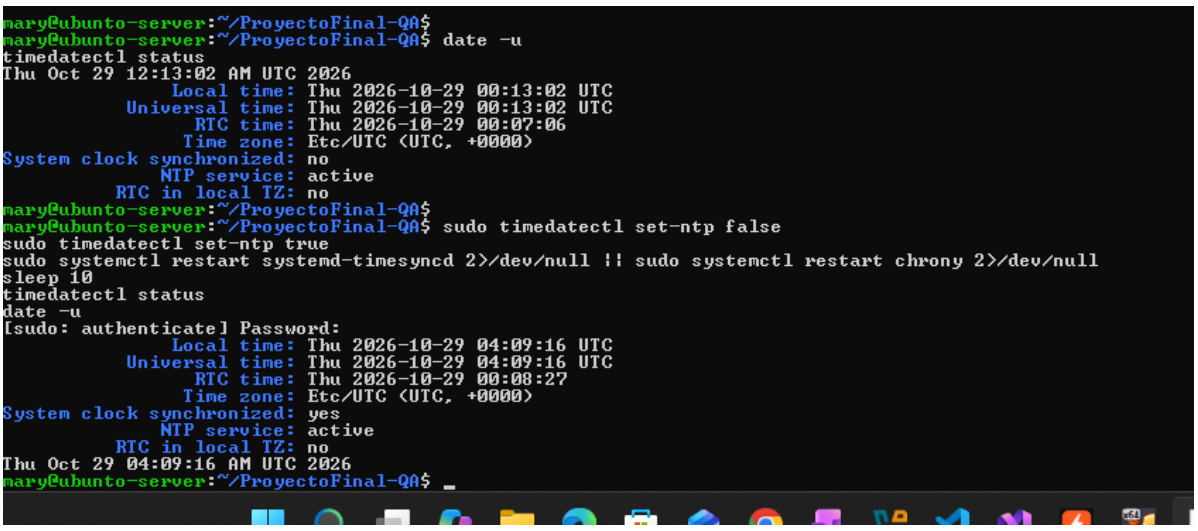
Terraform mostrando el plan: qué va a crear en Google Cloud (red, firewall, IP y máquina virtual).



Ansible instalando Docker y la tienda dentro de la máquina en la nube.



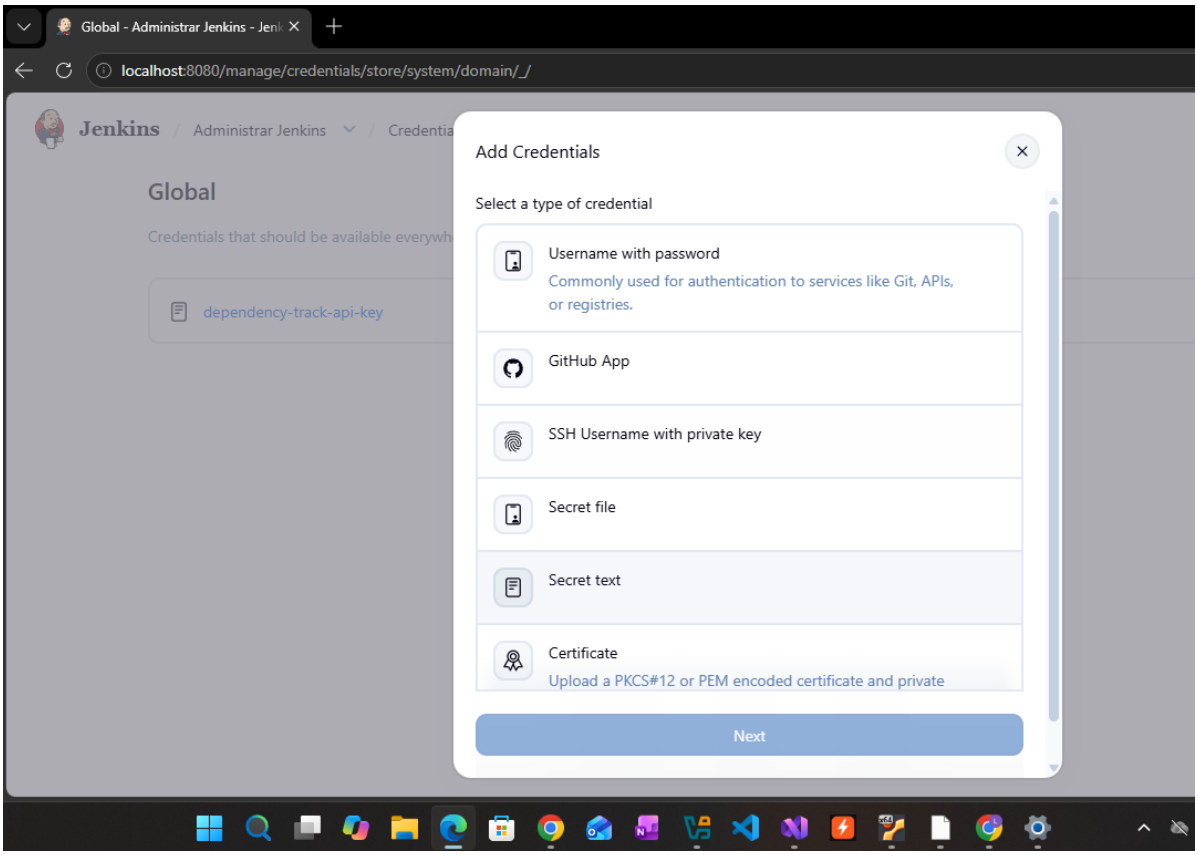
Un problema que tuvimos: el reloj de la máquina virtual estaba atrasado y Google rechazaba la conexión. Lo arreglamos sincronizando la hora.



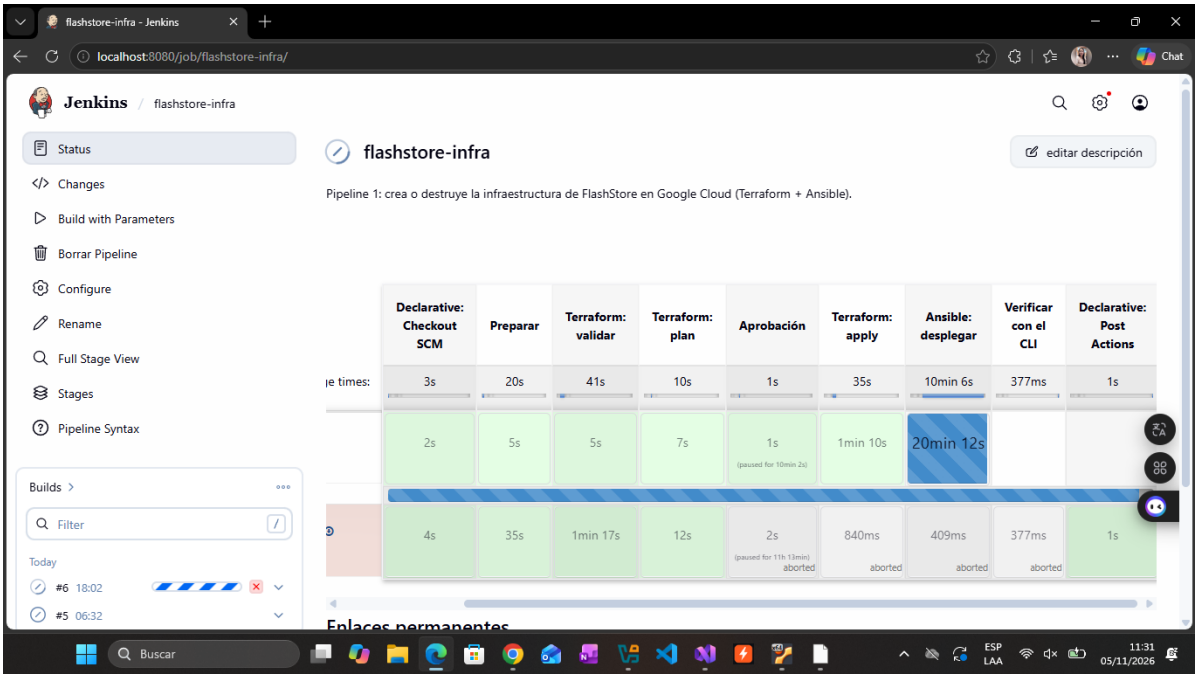
8. Pipelines en Jenkins

Armamos dos pipelines. El primero crea o destruye la infraestructura. El segundo prueba todo y decide GO o NO-GO.

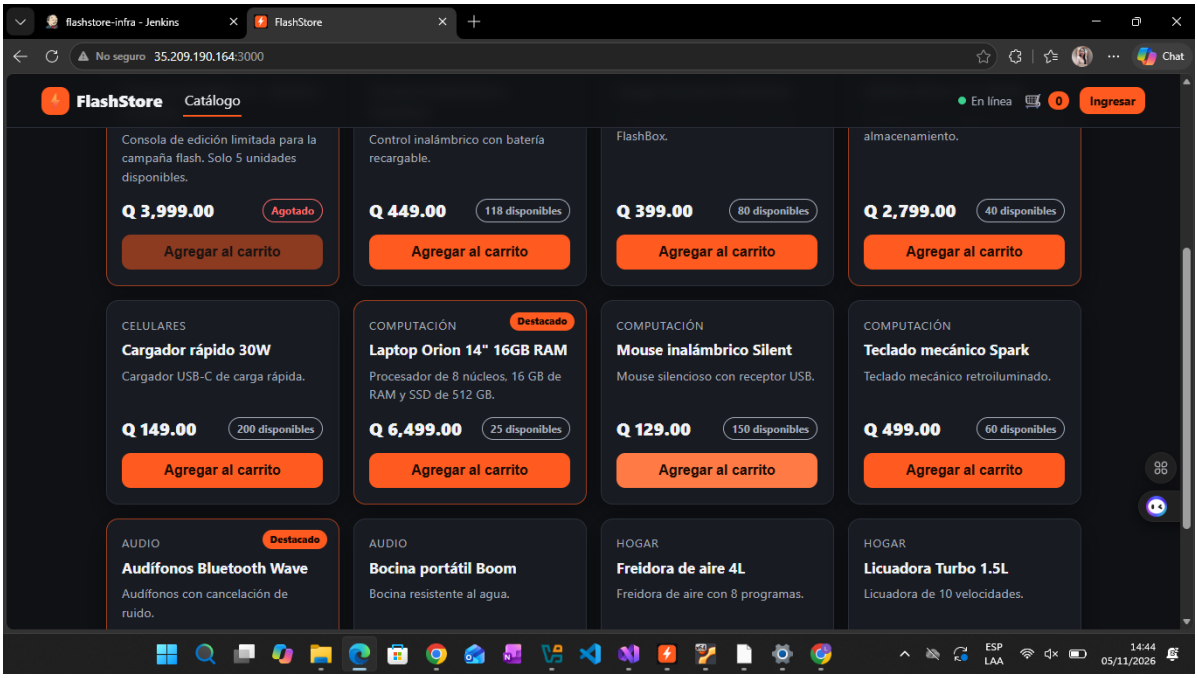
Guardamos las llaves y contraseñas en las credenciales de Jenkins, nunca en el código.



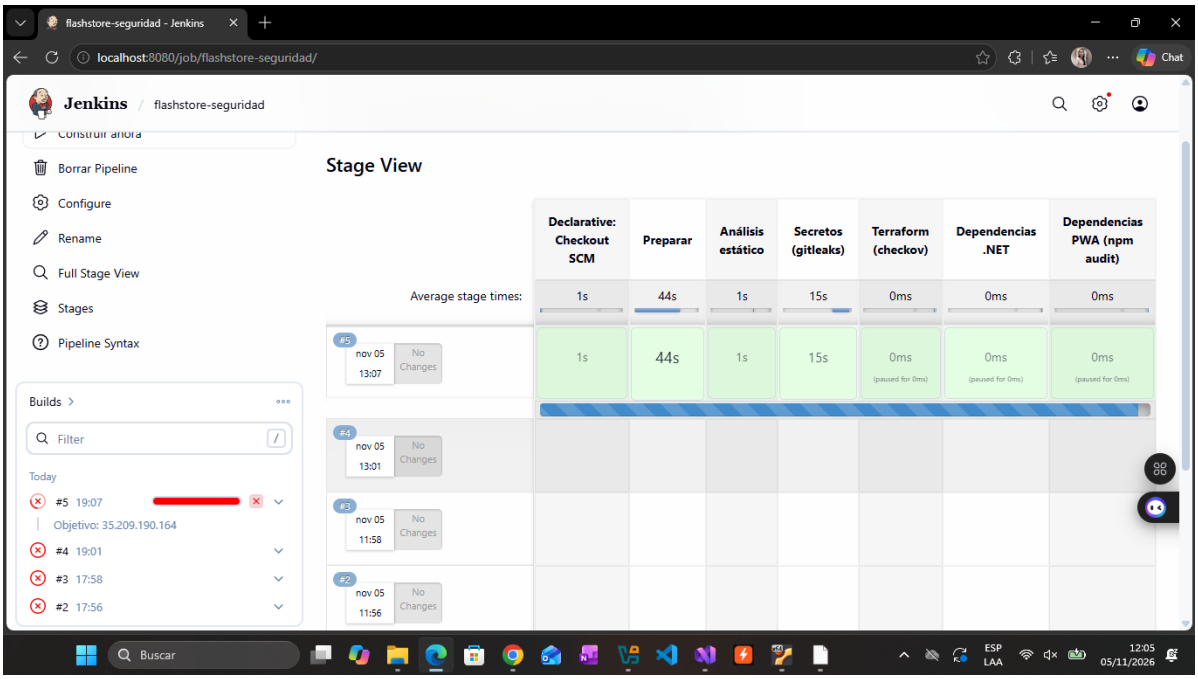
Pipeline 1 (flashstore-infra) completo: valida, planifica, espera aprobación, crea la infraestructura, despliega con Ansible y verifica.



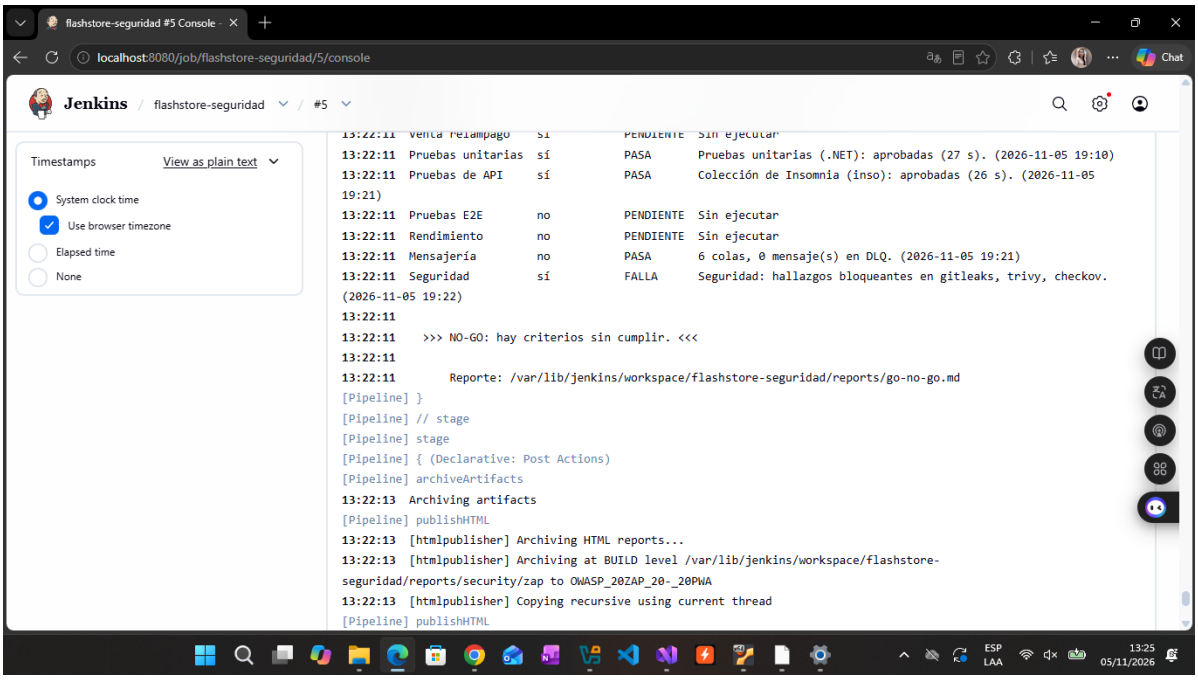
La tienda funcionando en la IP pública de Google Cloud.



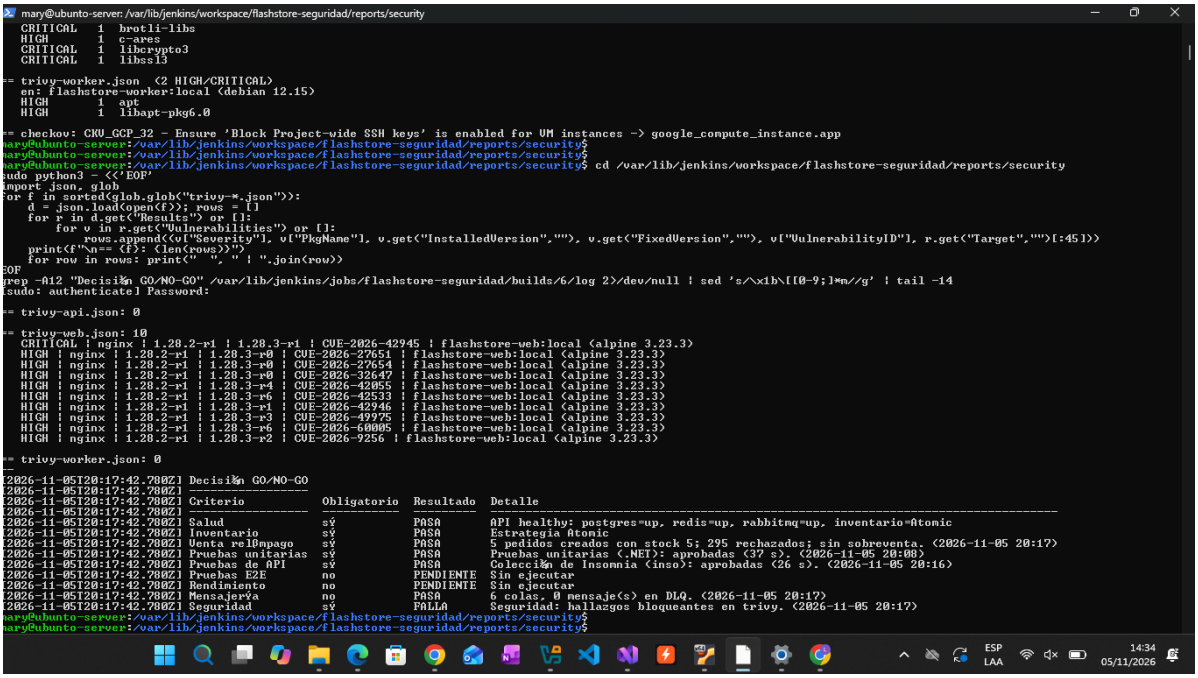
Pipeline 2 (flashstore-seguridad) corriendo sus etapas.



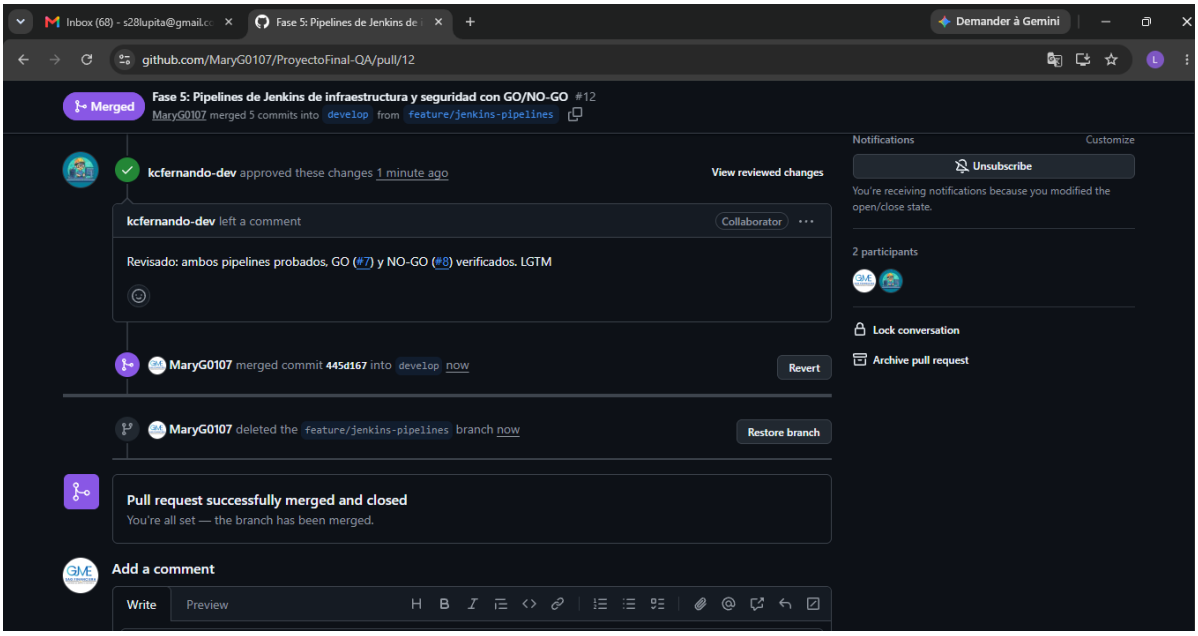
Primer NO-GO del pipeline 2: había criterios sin cumplir.



trivy encontró vulnerabilidades en las imágenes Docker. Cambiamos a imágenes más seguras hasta llegar a 0.



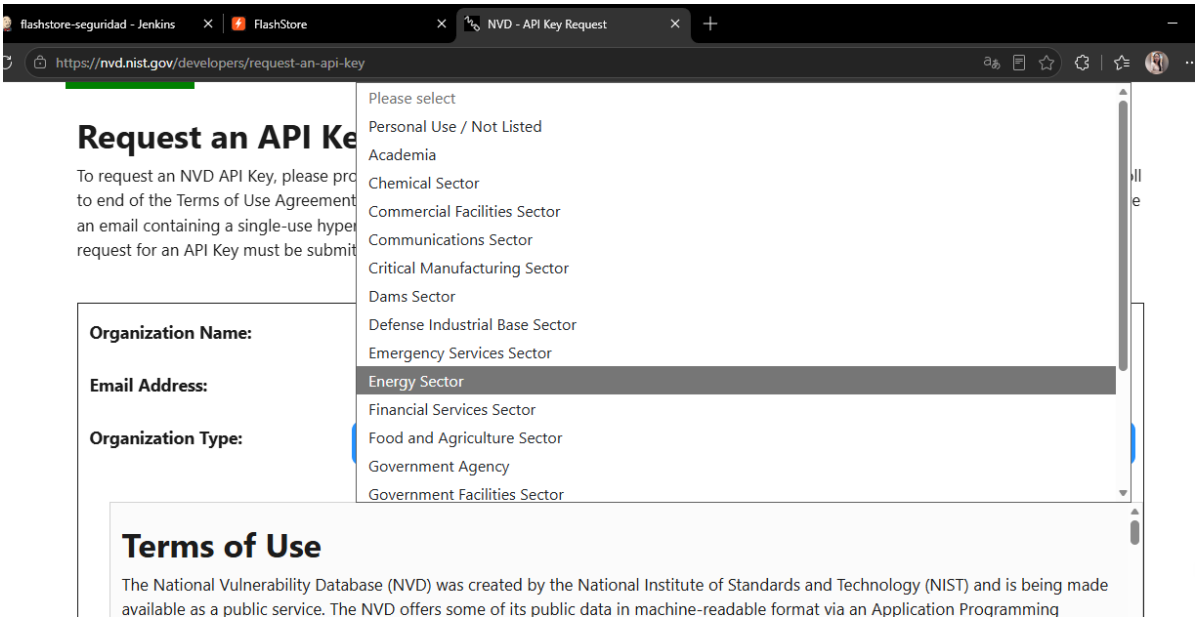
Pull request de la fase 5 (Jenkins) aprobado y unido.



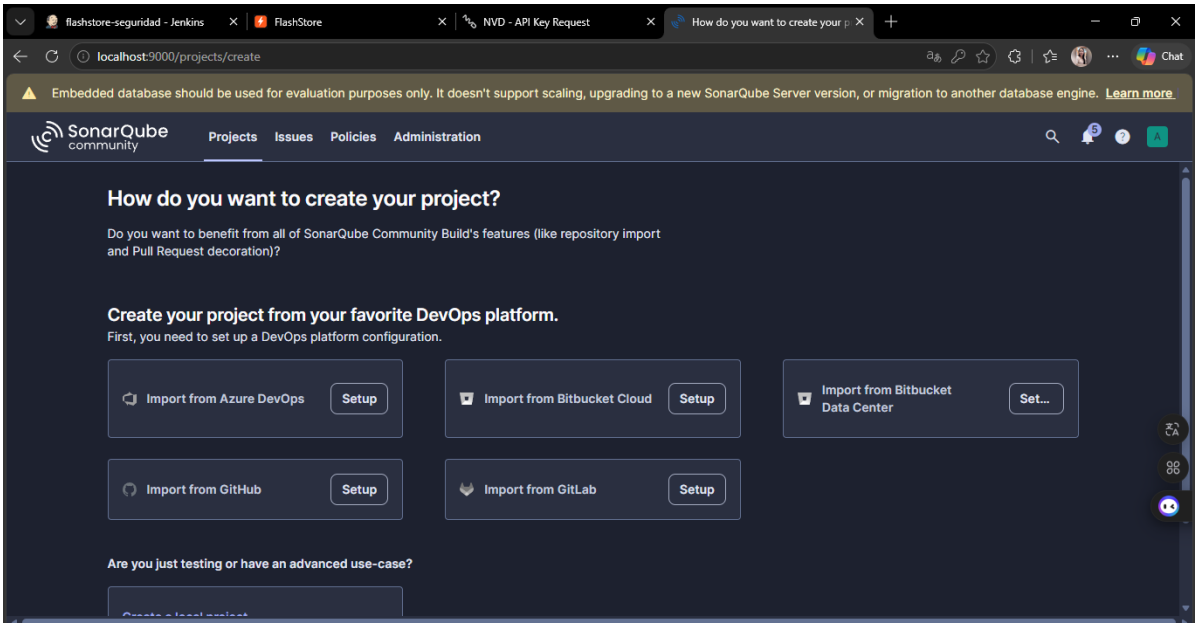
9. Más herramientas en el pipeline 2

Agregamos lo que pedía el enunciado: SonarQube, OWASP Dependency-Check, Semgrep (en lugar de Fortify), Cypress y k6.

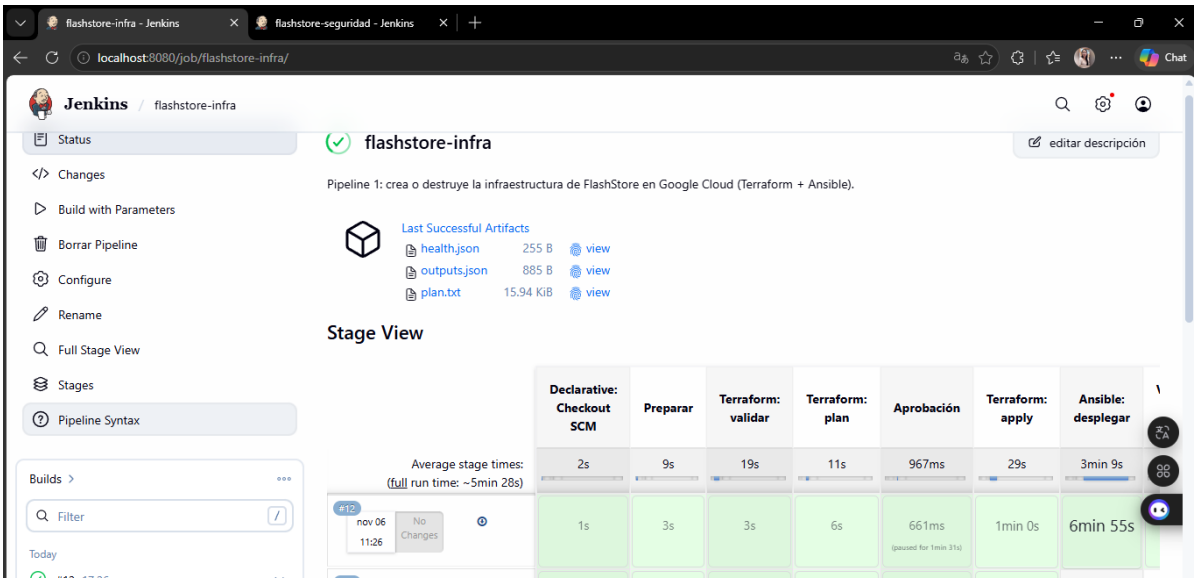
Solicitud de la llave gratuita de NVD para Dependency-Check. El correo nunca llegó, así que descargamos la base de datos completa sin llave.



SonarQube instalado en la máquina virtual para revisar la calidad del código.



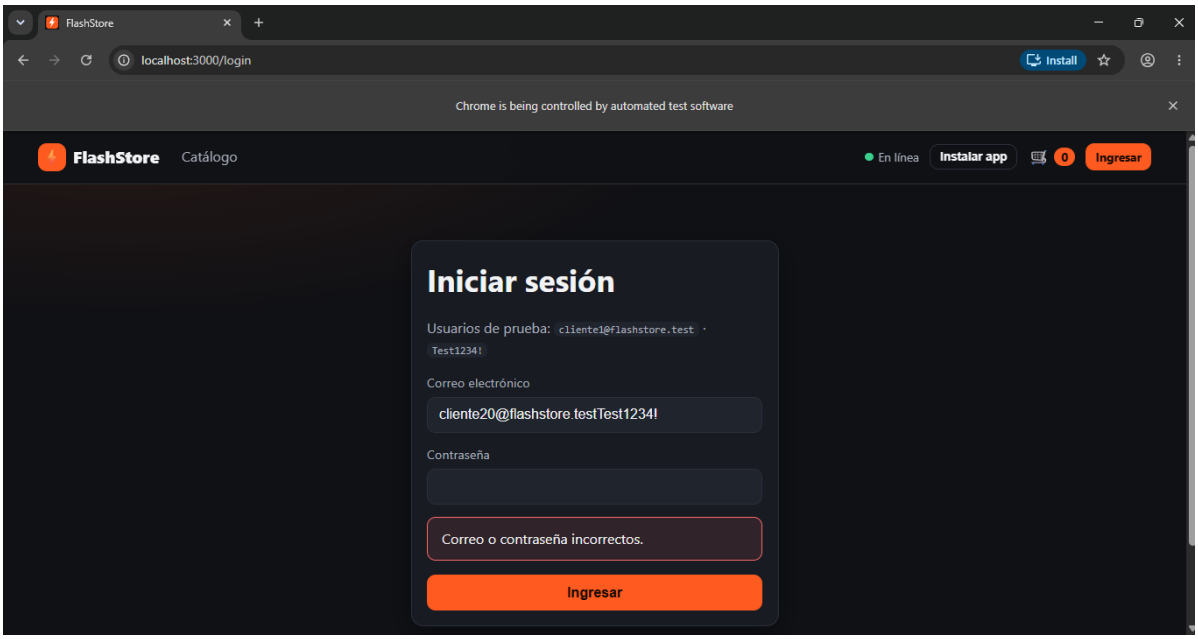
Pipeline 1, build #12, en verde con los cupones ya desplegados.



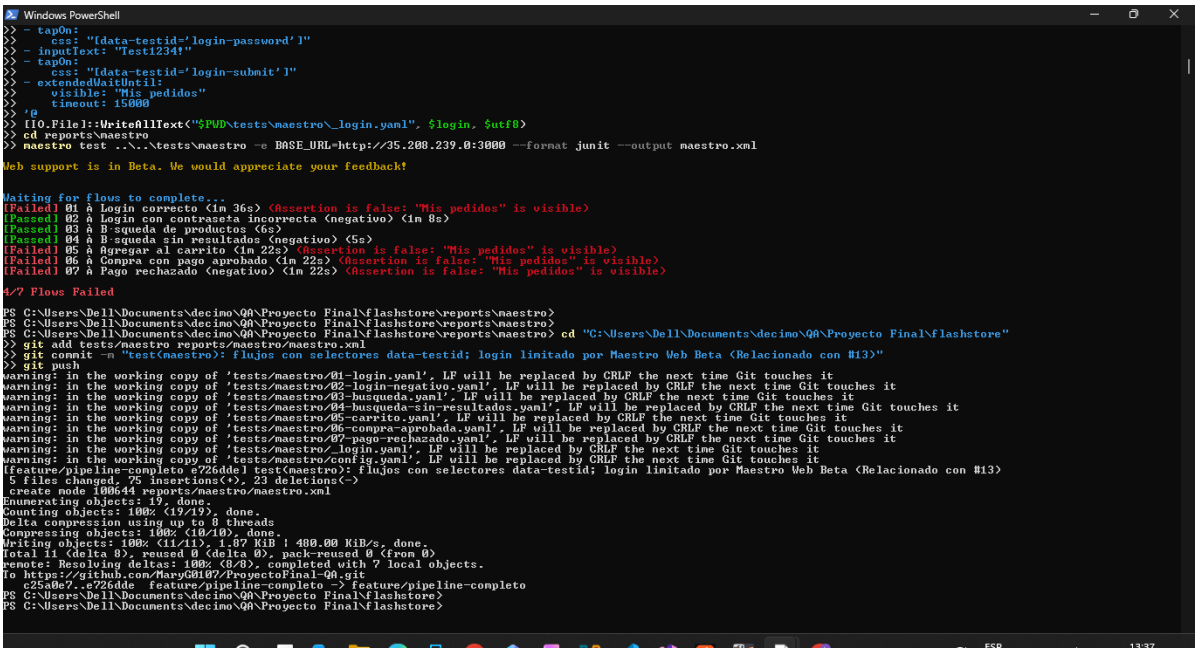
10. Pruebas móviles con Maestro

Maestro Web todavía está en versión Beta. Cuando escribe el correo, el cursor no pasa a la casilla de la contraseña, así que los flujos que necesitan iniciar sesión fallan. No es un error de la tienda: esos mismos flujos los prueba Cypress en el pipeline.

Maestro escribió el correo y la contraseña juntos en la misma casilla. La tienda respondió bien: "Correo o contraseña incorrectos".



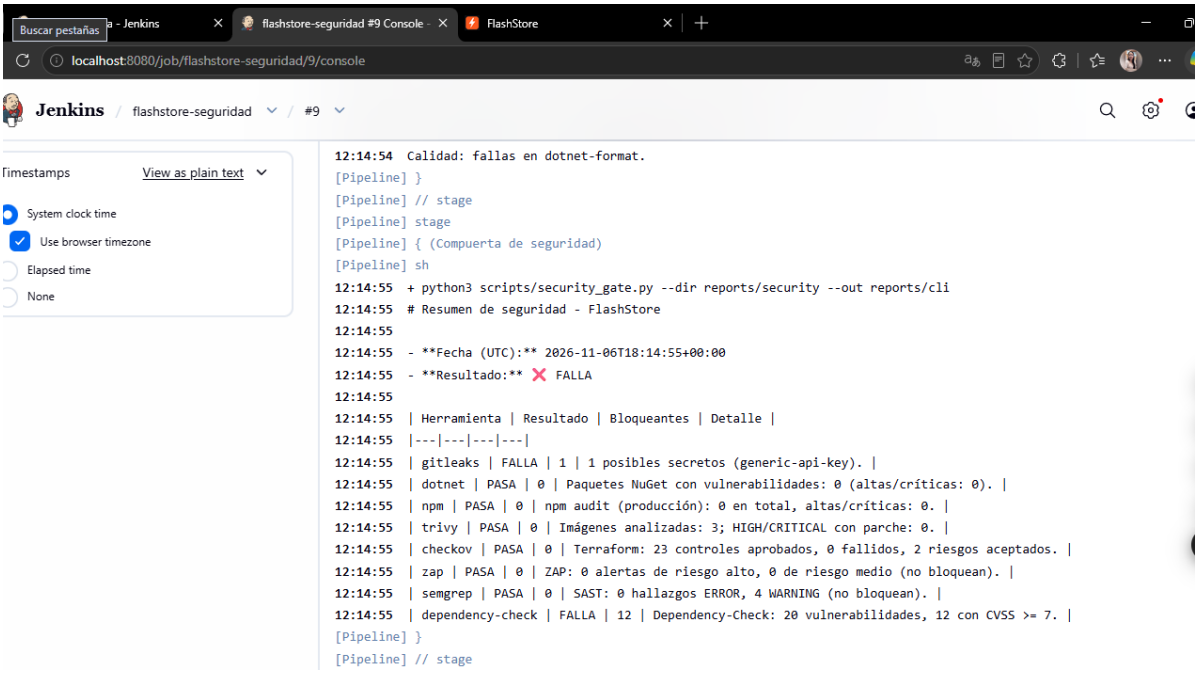
Resultado final de Maestro: pasan 3 de 7 flujos. Queda documentado como limitación de la herramienta.



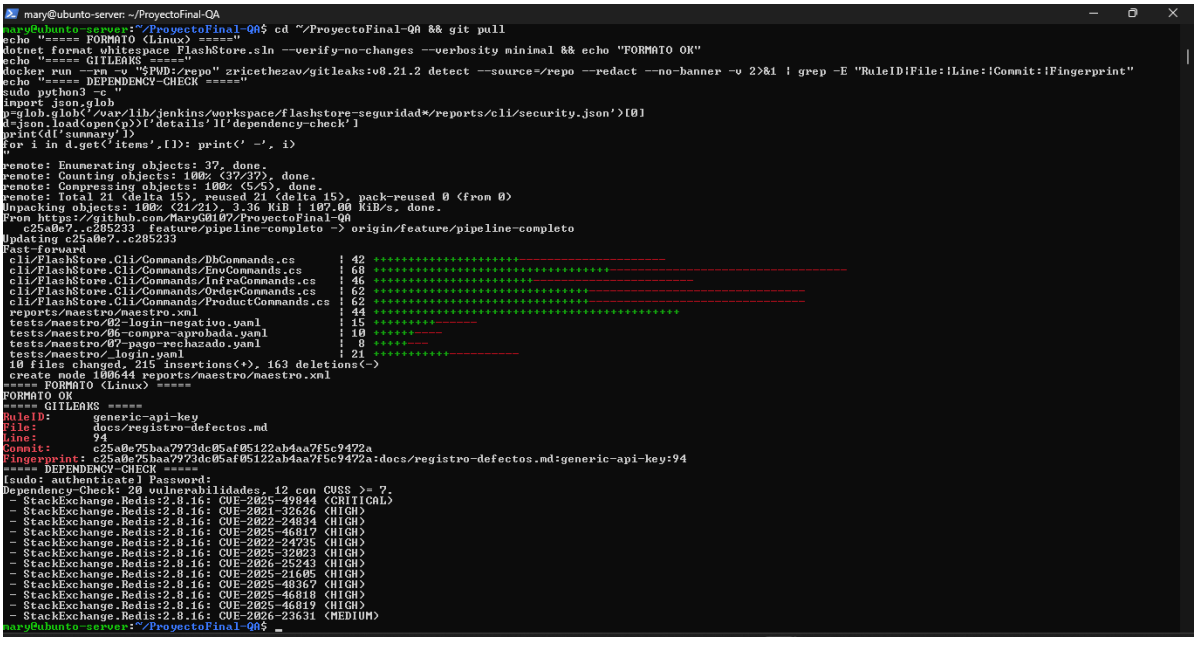
11. El NO-GO del build #9 y cómo lo resolvimos

Antes del GO final, el pipeline encontró tres cosas. Las revisamos una por una sin bajar ninguna regla.

Build #9: falla la seguridad por gitleaks (1 posible secreto) y Dependency-Check (12 alertas).



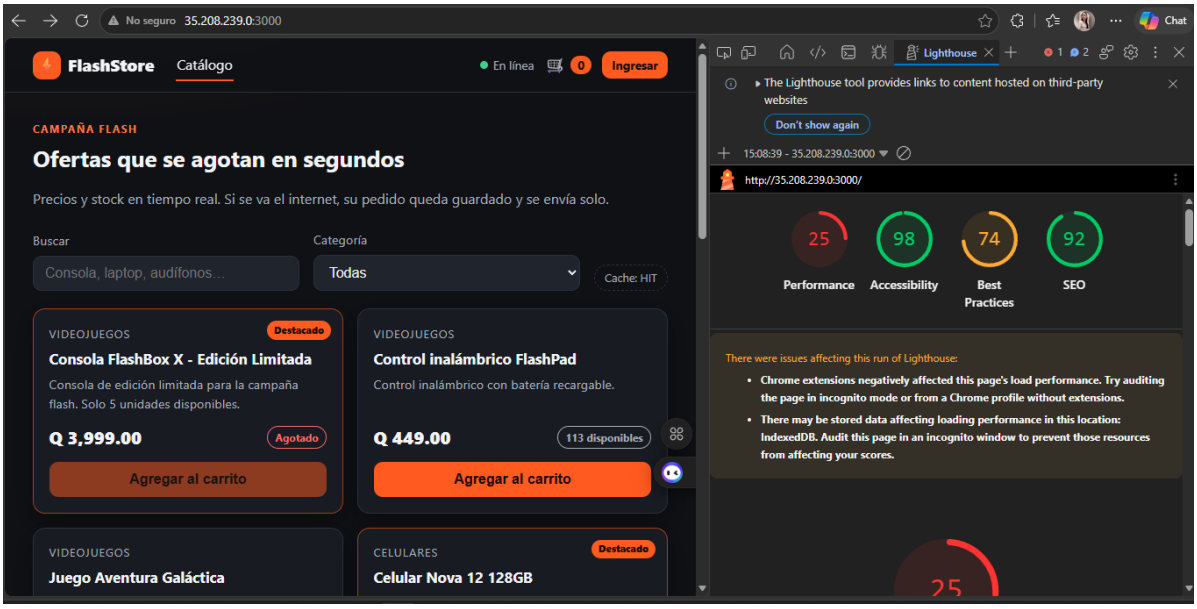
Revisamos los detalles. El "secreto" era un texto de ejemplo copiado en la documentación. Las 12 alertas eran de la librería cliente de Redis, que Dependency-Check confundió con el servidor Redis. Las dos eran falsas alarmas, y las anotamos con su justificación.



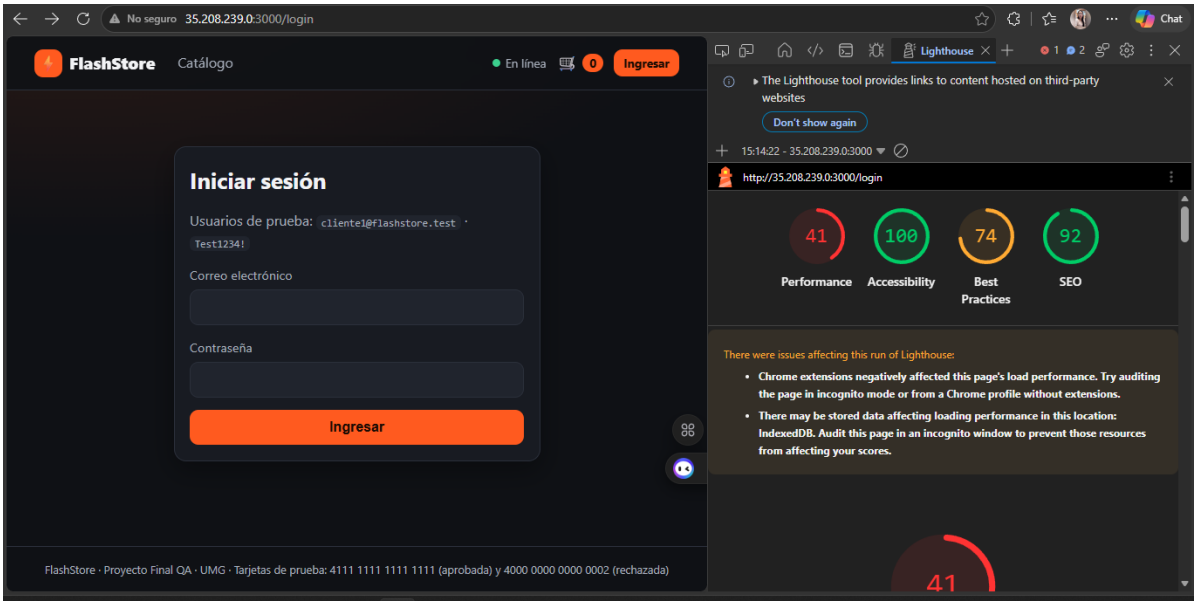
12. Accesibilidad con Lighthouse

Lighthouse viene dentro del navegador. Analiza la página y le pone notas del 0 al 100 en rendimiento, accesibilidad, buenas prácticas y SEO. Lo corrimos en modo celular sobre la tienda publicada en Google Cloud.

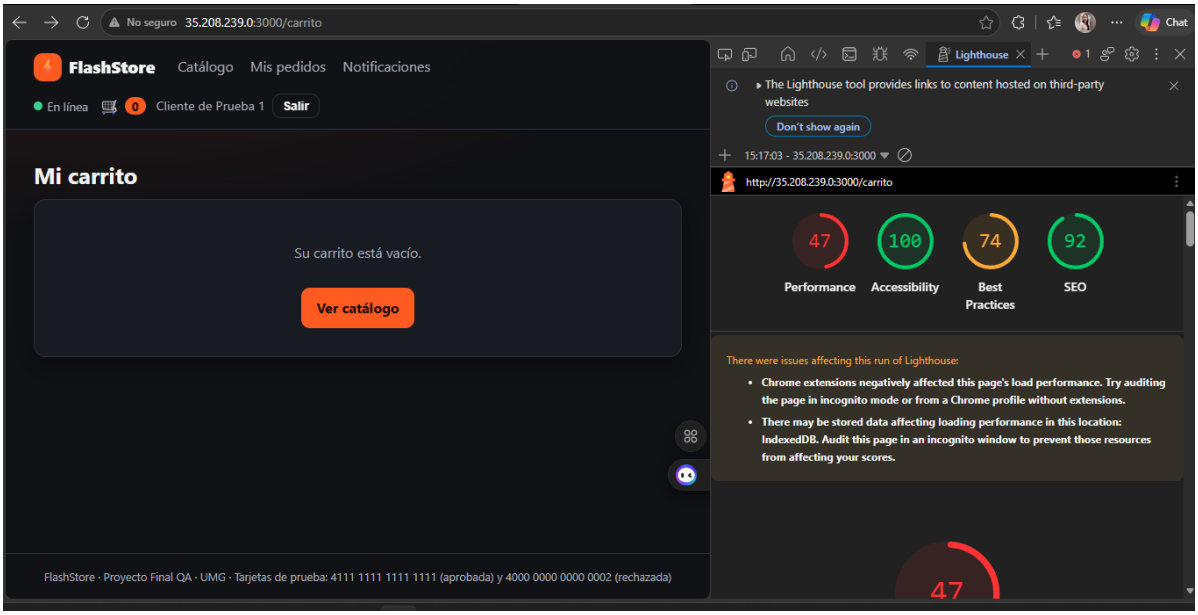
Catálogo: accesibilidad 98, buenas prácticas 74, SEO 92 y rendimiento 25.



Inicio de sesión: accesibilidad 100, buenas prácticas 74, SEO 92 y rendimiento 41.



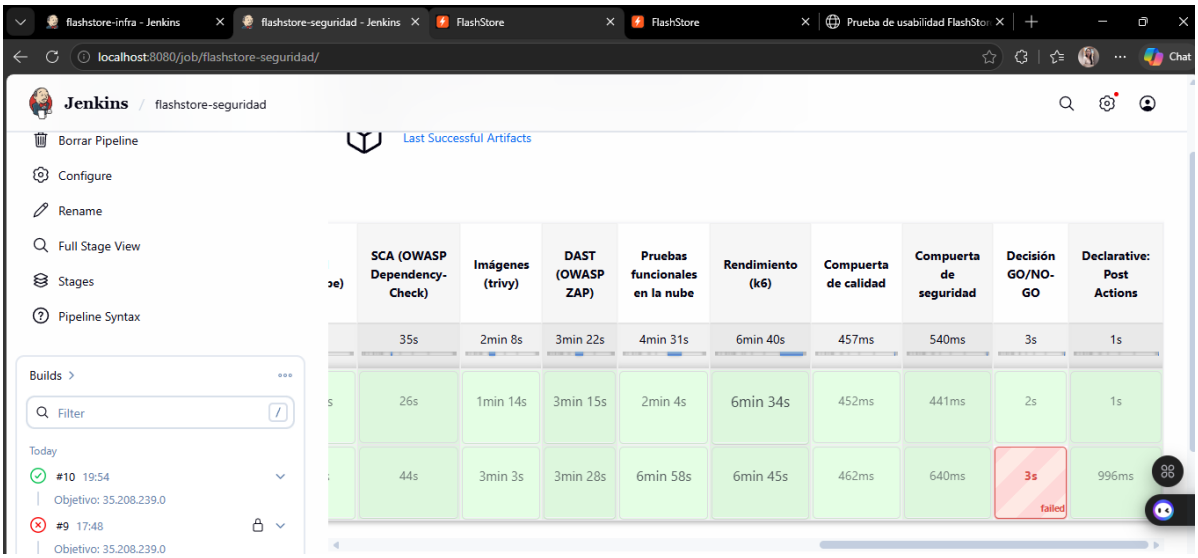
Carrito, con la sesión iniciada: accesibilidad 100, buenas prácticas 74, SEO 92 y rendimiento 47. El cuadro amarillo es el aviso de que las extensiones del navegador afectaron el rendimiento.



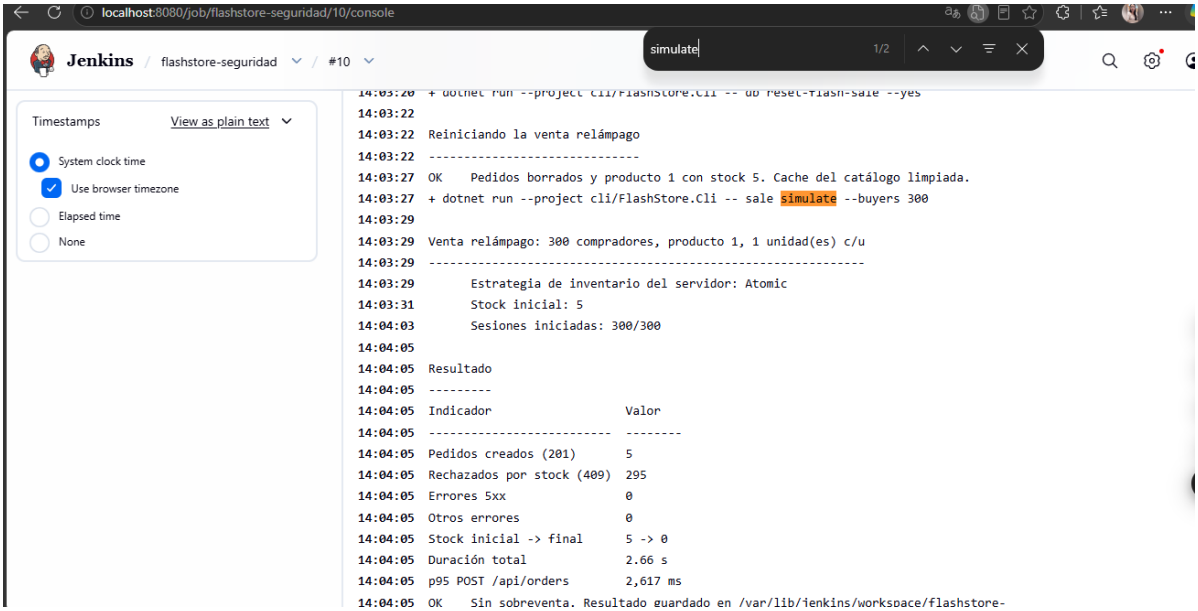
13. El GO final (build #10)

Con todo corregido, volvimos a correr el pipeline 2.

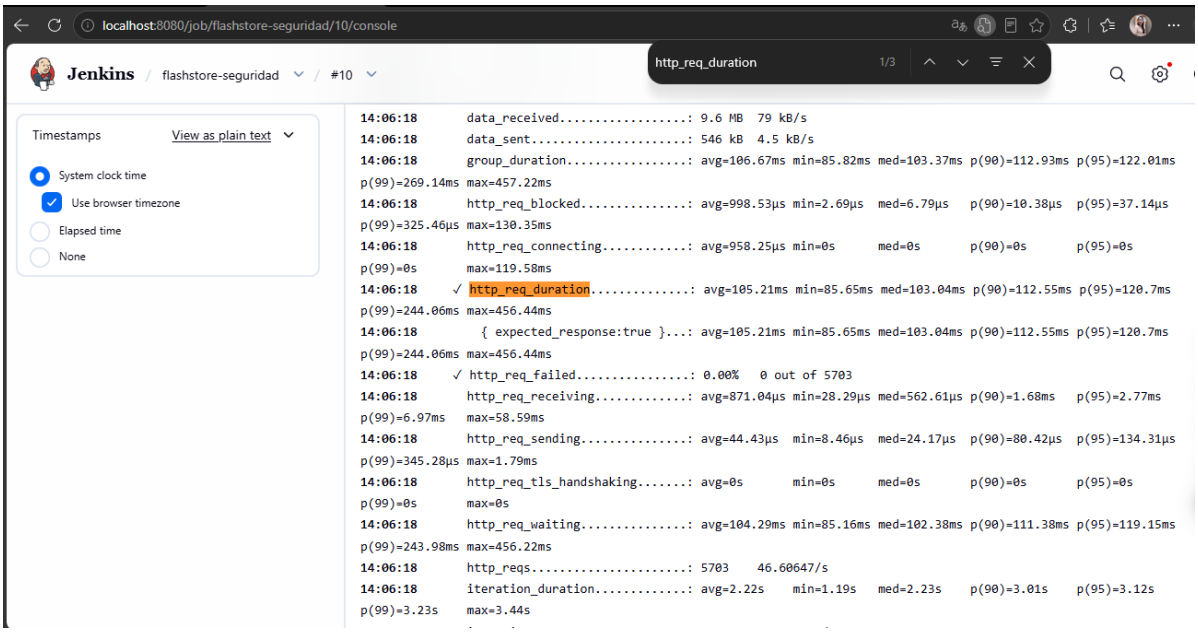
Arriba el build #10 en verde y abajo el #9 en rojo. Se ve la diferencia en la última etapa, la decisión GO/NO-GO.



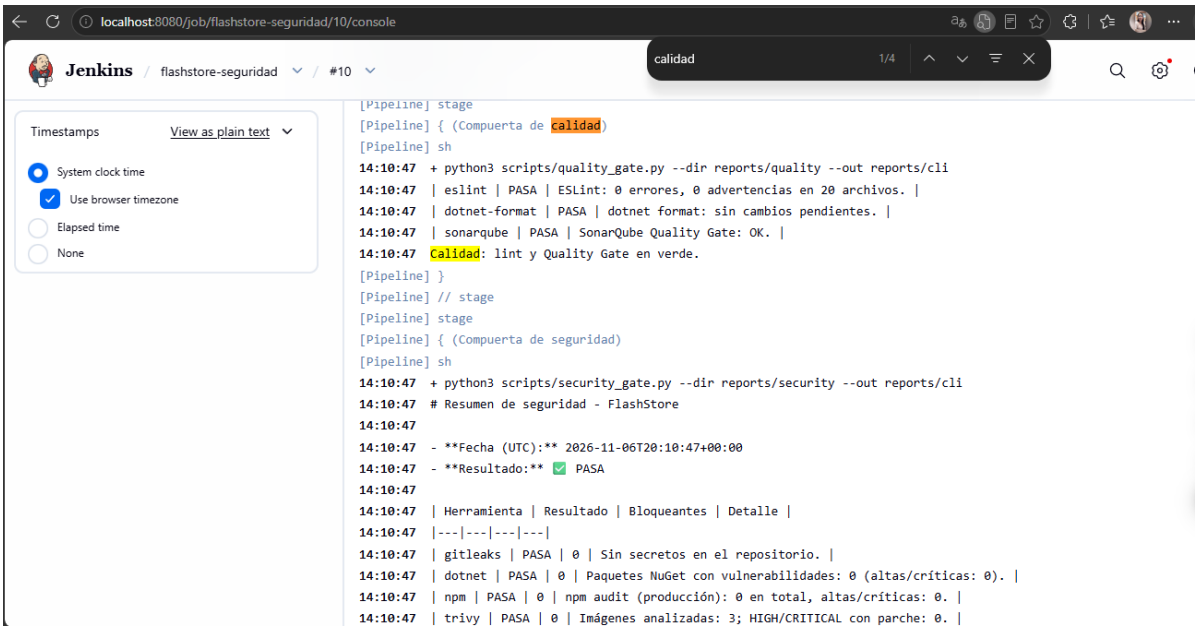
Venta relámpago: 300 compradores, 5 pedidos creados, 295 rechazados por falta de stock y ningún error. Sin sobreventa.



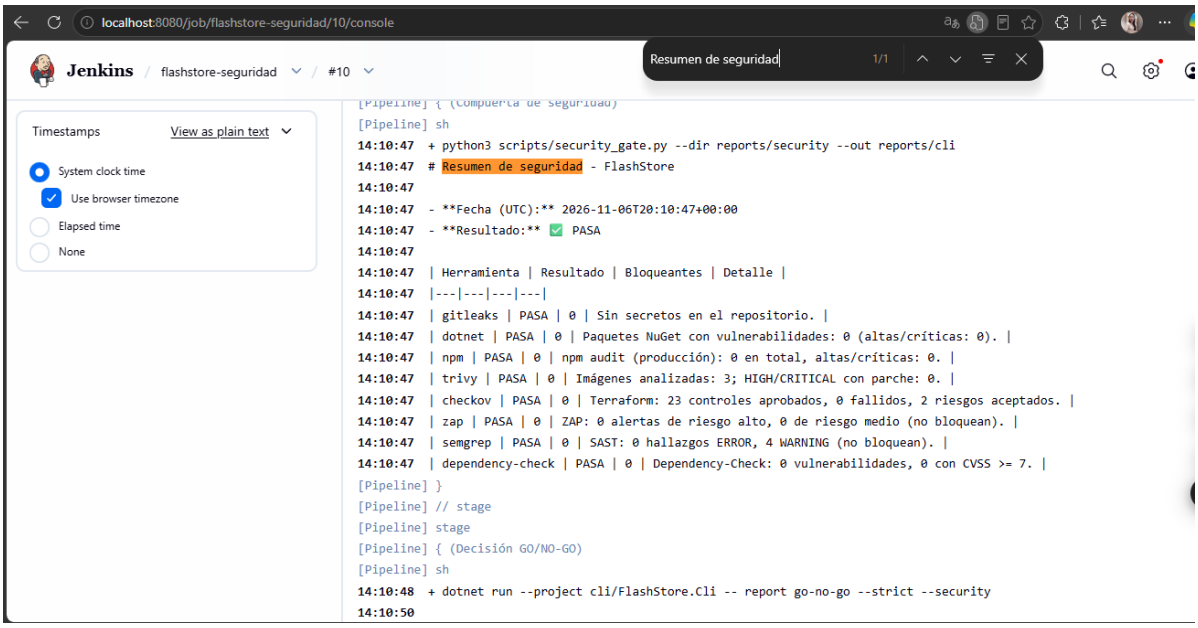
Prueba de rendimiento con k6: 5 703 solicitudes, 0 % de fallas y el 95 % respondió en menos de 121 ms.



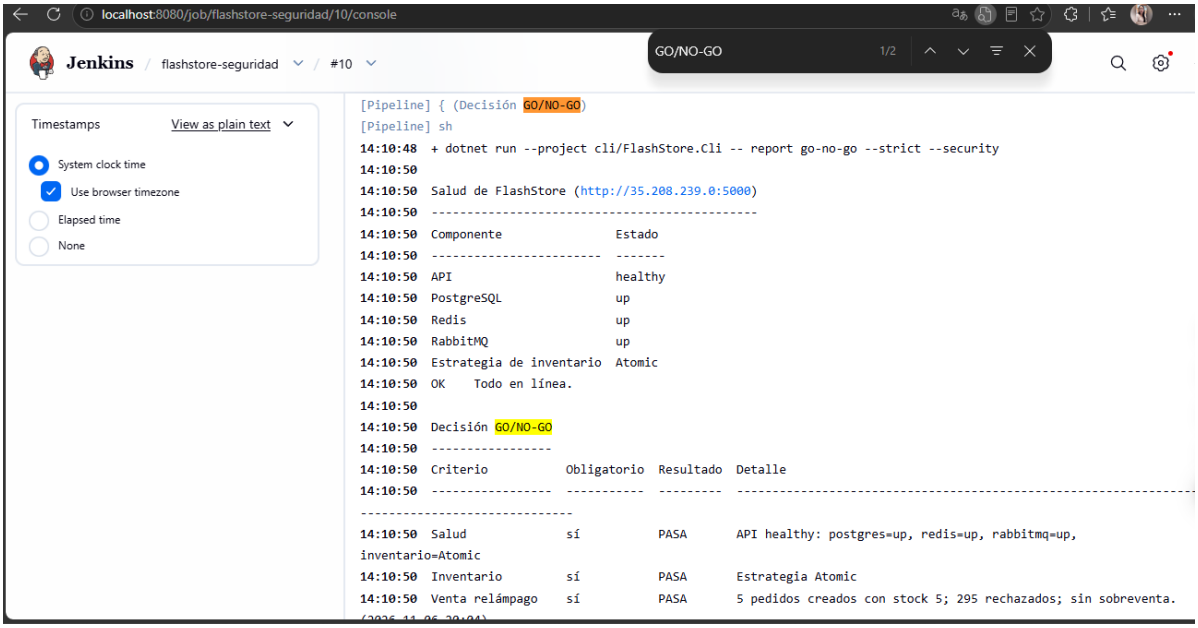
Compuerta de calidad: ESLint, dotnet format y SonarQube en verde.



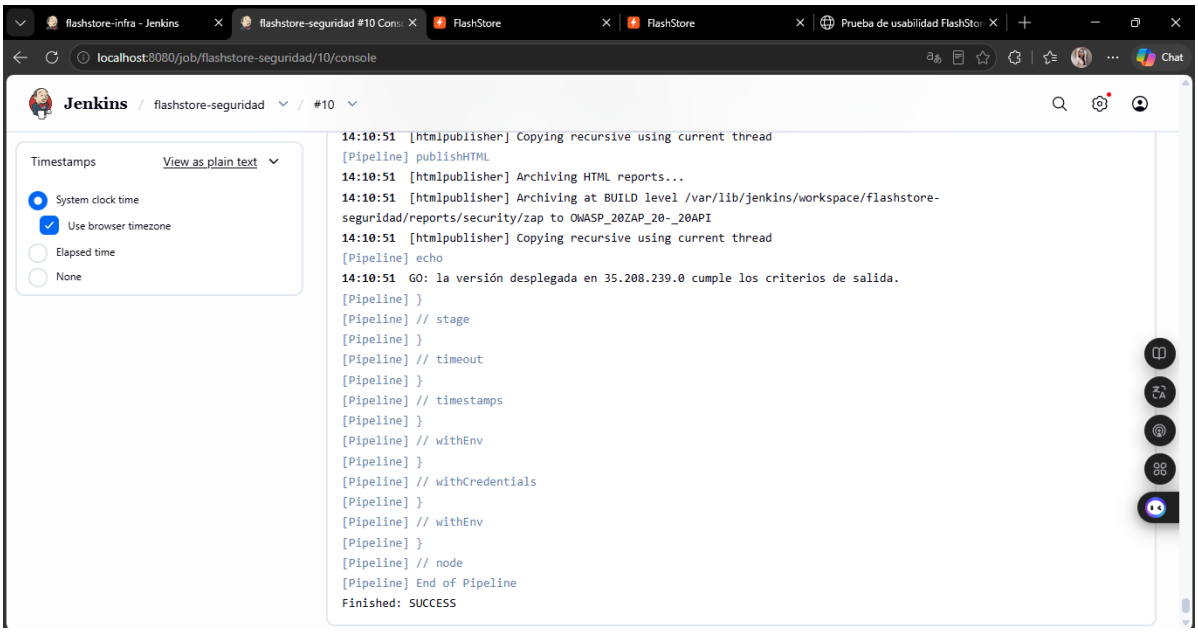
Compuerta de seguridad: las 8 herramientas pasan con 0 hallazgos bloqueantes.



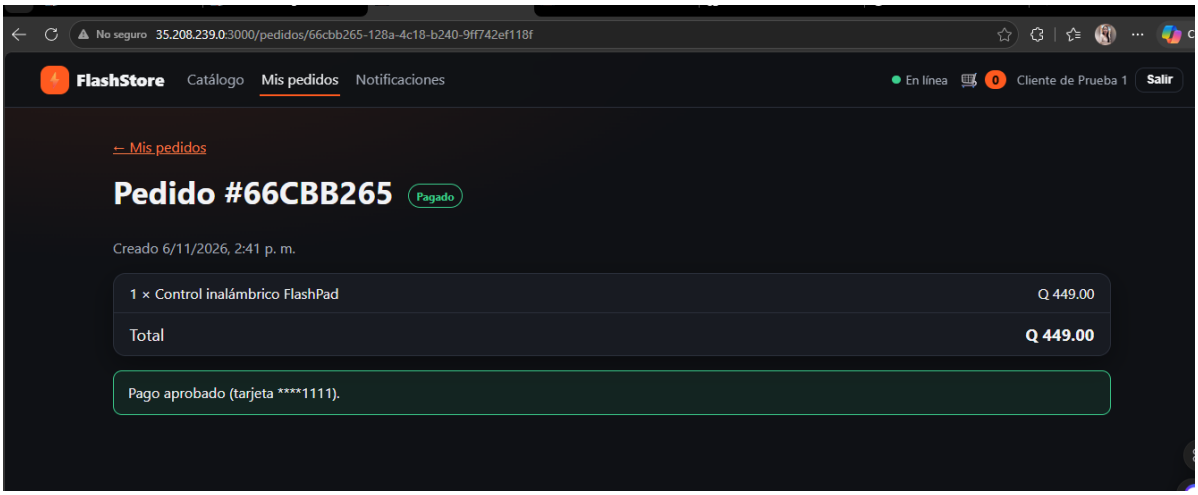
La decisión: todos los componentes en línea, estrategia Atomic y venta sin sobreventa.



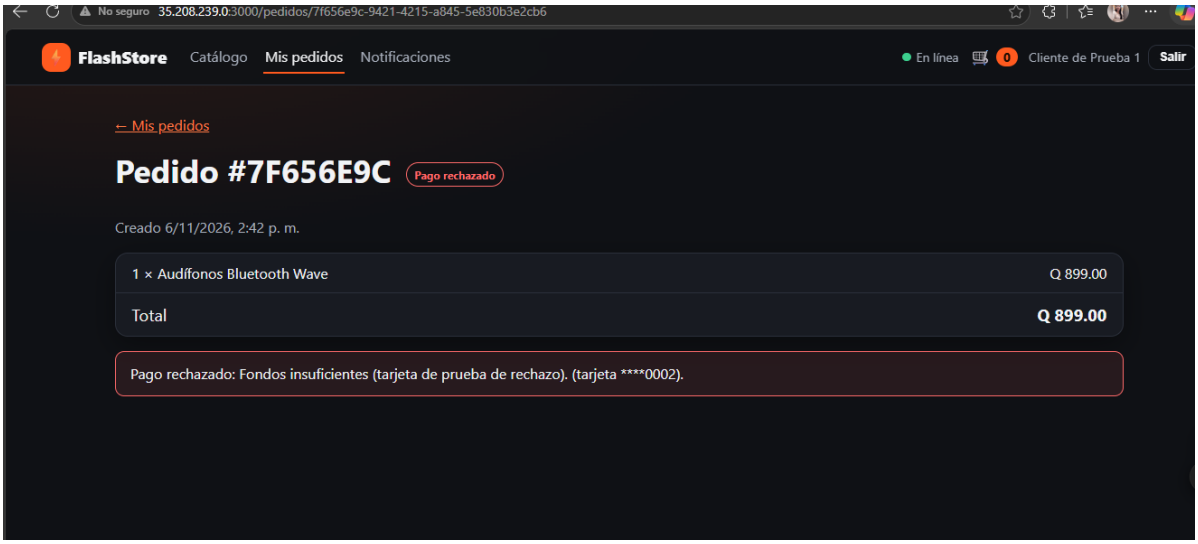
El final del build #10: "GO: la versión desplegada cumple los criterios de salida".



Prueba manual en la nube: compra con la tarjeta aprobada.



Prueba manual en la nube: compra con la tarjeta rechazada, con un mensaje claro.



14. Resultados en números

Venta relámpago

Indicador	Valor
Compradores	300
Pedidos creados	5
Rechazados por stock	295
Errores del servidor	0
Stock inicial y final	5 → 0
Tiempo del 95 % de los pedidos	2.6 s

Los 300 pedidos llegan al mismo tiempo y todos tocan la misma fila del inventario, por eso esperan su turno. Es más lento que la meta de 0.8 s del plan, pero así garantizamos que no haya sobreventa. Lo dejamos anotado como riesgo para mejorar.

Rendimiento (k6)

Métrica	Valor
Solicitudes	5 703
Fallas	0 %
Promedio	105 ms
95 % por debajo de	121 ms
La más lenta	456 ms

Seguridad: build #9 contra build #10

Herramienta	#9	#10
gitleaks	1 (falsa alarma)	0
NuGet (dotnet)	0	0
npm audit	0	0
trivy	0	0
checkov	0 fallidos	0 fallidos
OWASP ZAP	0 altas	0 altas
Semgrep	0 errores	0 errores
Dependency-Check	12 (falsa alarma)	0

Usabilidad

Participante	Tareas logradas	Puntaje SUS
P1	5 de 5	75 (meta: 68 o más)

Fue una prueba piloto con una persona. Para que el resultado sea más confiable, el plan pide 5 personas.

Accesibilidad (Lighthouse)

Página	Rendimiento	Accesibilidad	Buenas prácticas	SEO
Catálogo	25	98	74	92
Login	41	100	74	92

Carrito	47	100	74	92

La meta del plan era accesibilidad de 90 o más, y las tres páginas la cumplen (98, 100 y 100). El rendimiento sale bajo porque Lighthouse simula un celular lento con red 4G, el servidor está en Estados Unidos y el propio Lighthouse avisó que las extensiones del navegador afectaron la medición. Para mejorarlo se puede comprimir y dividir el código de la PWA y servir los archivos desde un CDN.

15. Problemas encontrados

ID	Qué pasó	Cómo se resolvió
DEF-00	Sobreventa con la estrategia Unsafe (a propósito)	Estrategia Atomic
DEF-01	Contraseñas visibles en el registro de Jenkins	Se ocultaron y se cambiaron todas
DEF-02	Jenkins se quedaba esperando después de Ansible	Ajuste en la conexión SSH
DEF-03	65 vulnerabilidades en las imágenes Docker	Imágenes más seguras: 0
DEF-04	Una regla de checkov fallaba en Terraform	Se corrigió el valor
DEF-05	La venta relámpago corría sin compradores	Valor por defecto de 300
DEF-06	Archivos del CLI con mal formato	Se aplicó dotnet format
FP-01/02	gitleaks marcó un texto de ejemplo como secreto	Falsa alarma, ignorada solo esa línea
FP-03	Dependency-Check confundió la librería de Redis	Falsa alarma, suprimida con justificación
INC-01 a 05	Problemas del ambiente: reloj, Windows, Node, rama de Jenkins y Maestro Beta	Documentados con su solución

16. Riesgos que quedan

- La máquina en la nube tiene IP pública; el firewall solo deja entrar a nuestras redes.
- Dependency-Check usa una base descargada el 6 de noviembre porque no llegó la llave de NVD.
- Todo corre en una sola máquina, sin respaldo si se cae.
- La venta relámpago tarda 2.6 s en el peor momento; se puede mejorar con una cola en Redis.
- La prueba de usabilidad fue con una sola persona.
- El rendimiento de la página en celular (Lighthouse) es bajo: 25 a 47.
- Maestro Web está en Beta; conviene repetir con Android.

17. Lo que aprendimos

- Que el pipeline diga NO-GO también es un buen resultado: significa que está haciendo su trabajo.
- Hay que leer los registros, no solo ver si salió verde. Así encontramos las contraseñas visibles en Jenkins.
- Una alerta no siempre es un problema real, pero hay que revisarla y justificarla antes de ignorarla.
- No conviene depender de una herramienta en Beta para decidir si algo sale o no.
- El ambiente también falla (el reloj, Windows, versiones) y documentarlo ahorra tiempo la próxima vez.

18. Conclusión

FlashStore cumple el objetivo principal: aunque 300 personas compren al mismo tiempo, nunca vende más de lo que hay. El pipeline de Jenkins revisa la seguridad, la calidad, el rendimiento y las funciones, y decide solo si la versión puede salir. Después de corregir y justificar los hallazgos, el build #10 dio GO.